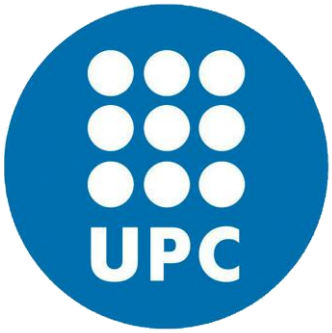
UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Escola Politècnica Superior d'Edificació de Barcelona

GRADO EN INGENIERÍA EN GEOINFORMACIÓN Y GEOMÁTICA

TRABAJO DE FIN DE GRADO



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Politècnica Superior d'Edificació de Barcelona

DISEÑO Y DESARROLLO DE UNA IDE INTERACTIVA PARA LA PROMOCIÓN DEL PATRIMONIO CULTURAL Y LA DESCENTRALIZACIÓN DEL TURISMO EN CATALUNYA

Autor: Guillem Fernández Arrondo

Director: Juan Carlos González González

Curso 2025/2026

Resumen

El punto de partida de este trabajo es dar respuesta a un problema concreto: en Catalunya hay una cantidad enorme de información cultural disponible (patrimonio histórico, fiestas, eventos, etc.), pero está dispersa. La Generalitat la publica en formatos de datos abiertos, pero en tablas crudas sin ninguna capa de visualización. No existe ninguna plataforma que muestre a la vez dónde está el patrimonio y qué ocurre culturalmente cerca.

Este proyecto construye esa plataforma. El objetivo es diseñar y desarrollar una Infraestructura de Datos Espaciales (IDE) interactiva que integre en un mismo visor el patrimonio monumental estático y la agenda cultural dinámica de Catalunya, usando únicamente tecnologías de código abierto y datos públicos.

Primero he construido dos *pipelines ETL*[1] en Python: uno extrae los eventos culturales activos desde la API Socrata de la Generalitat, y el otro barre los 947 municipios de Catalunya lanzando consultas SPARQL a Wikidata con un radio de 16 km por municipio para capturar el patrimonio monumental. Todo esto se carga en una base de datos PostgreSQL con PostGIS bajo el esquema `ide_cultural`. Sobre esa base de datos he montado una *API REST* con *FastAPI* que sirve los datos en formato *GeoJSON* filtrados por el área visible en pantalla. El visor web, construido con *Leaflet.js* y JavaScript, consume esa API y ofrece filtros dinámicos de texto, categoría y fecha, clustering de marcadores, geolocalización, análisis de proximidad y un generador automático de rutas culturales.

Los resultados son los siguientes. La base de datos tiene 31.007 monumentos extraídos, lo que representa una cobertura del 82,7% respecto al Inventari del Patrimoni Arquitectònic de Catalunya (IPAC). La API responde en menos de 300 ms en condiciones normales de uso, cuando el usuario tiene una zona concreta a la vista en vez de todo el territorio. El *endpoint* `/api/metadata` publica los metadatos del sistema según el Núcleo Español de Metadatos (NEM/ISO 19115), lo que hace la plataforma potencialmente integrable con catálogos institucionales como la IDEC.

Las limitaciones más importantes son tres: la Generalitat no publica los eventos con antelación, así que fiestas mayores recurrentes no aparecen hasta semanas antes de celebrarse; el 17,3% del IPAC no está documentado en Wikidata con los identificadores que el sistema usa como criterio de entrada; y el visor solo corre en local, sin despliegue en producción. Como mejoras futuras propongo integrar el Patrimoni Festiu de Catalunya como capa complementaria, desplegar el sistema en un servidor público y exponer servicios OGC estándar mediante *pygeoapi*.

[1] Secuencia encadenada de procesos automatizados de Extracción, Transformación y Carga de datos (ETL, del inglés Extract, Transform, Load).

Índice

Resumen 1
Índice 2
Índice figuras 4
Glosario 5
1. Introducción 8
1.1. Motivación y justificación: Origen y enfoque del proyecto 8
1.2. Objetivos 8
1.3. Alcance y metodología 9
2. Marco Teórico 10
2.1. Las Infraestructuras de Datos Espaciales (IDE) 10
2.2. Arquitectura Orientada a Servicios (SOA) 11
2.3. Estándares OGC y servicios de red 12
2.4. Metadatos: el NEM y la ISO 19115 13
2.5. Open Data y Turismo Regenerativo 14
3. Diseño del sistema 16
3.1. Requisitos funcionales y no funcionales 16
3.1.1. Requisitos funcionales 16
3.1.2. Requisitos no funcionales 17
3.2. Modelo de datos espacial en PostGIS 17
3.2.1. La tabla patrimoni_monumental 18
3.2.2. La tabla agenda_cultural 18
3.2.3. El esquema como unidad de organización 18
3.3. Diseño de la API REST 19
3.4. Diseño del frontend 19
3.4.1. Estructura narrativa y modo explorador 19
3.4.2. Gestión de marcadores y clustering 20
3.4.3. Motor de filtrado en el cliente 20
3.4.4. Análisis de proximidad y generador de rutas 20
4. Implementación 21
4.1. Pipeline ETL de la Agenda Cultural Dinámica 21
4.1.1. Extracción 21
4.1.2. Transformación 22
4.1.3. Carga: el patrón UPSERT 23

4.1.4. Purga de eventos caducados ... 23
4.1.5. Esquema de la tabla en PostGIS ... 24
4.2. Pipeline ETL del Patrimonio Estático ... 24
4.2.1. La red de nodos: obtención de los centros municipales ... 26
4.2.2. La consulta por radio: wikibase:around ... 26
4.2.3. Extracción en paralelo ... 27
4.2.4. Transformación y enriquecimiento ... 27
4.2.5. Carga a PostGIS ... 28
4.3. Auditoría de cobertura espacial ... 28
4.4. Implementación del visor web ... 29
4.4.1. Leaflet.js como librería de renderizado ... 29
4.4.2. Turf.js para el geoprocesamiento en cliente ... 29
4.4.3. Decisiones de experiencia de usuario ... 30
4.5. Aproximaciones descartadas y decisiones de rediseño ... 30
4.5.1. Primeras versiones del ETL ... 30
4.5.2. Buscando las fuentes correctas ... 31
4.5.3. El intento del Patrimoni Festiu de Catalunya ... 31
5. Resultados y Validación ... 32
5.1. Resultados de la extracción de datos ... 32
5.1.1. Capa de patrimonio monumental ... 32
5.1.2. Capa de agenda cultural ... 33
5.2. Validación funcional del visor ... 33
5.2.1. Rendimiento de la API ... 33
5.2.2. Filtros y carga dinámica ... 34
5.2.3. Clustering ... 34
5.3. Conformidad normativa ... 34
5.3.1. GeoJSON y RFC 7946 ... 34
5.3.2. NEM e ISO 19115 ... 34
6. Conclusiones ... 35
6.1. Evaluación de objetivos ... 35
6.2. Limitaciones del sistema ... 36
6.3. Mejoras futuras ... 36
6.4. Reflexión final ... 37
7. Análisis de sostenibilidad e implicaciones éticas ... 37
7.1. Impacto ambiental ... 37

7.2. Impacto económico 37
7.3. Impacto social 38
7.4. Implicaciones éticas 38
7.5. Relación con los ODS 38
Bibliografía 39
Anexo I: Repositorio del código 41

Índice figuras

Figura 1: Esquema conceptual de la arquitectura del sistema 10
Figura 2: Jerarquía institucional de referencia 11
Figura 3: Arquitectura SOA del sistema en tres capas 12
Figura 4: Distribución de turistas por municipio en Catalunya (2022) 15
Figura 5: Pipeline ETL de la Agenda Cultural Dinámica. 21
Figura 6: Pipeline ETL del Patrimonio Monumental. 25
Figura 7 - Mapa de cobertura del radar ETL: 947 buffers de 16 km sobre Catalunya 29
Figura 8 - Tablas resumen de la capa patrimoni_monumental 32

Glosario

API REST: Interfaz de programación que permite que dos sistemas se comuniquen a través de HTTP. En este proyecto, la API construida con FastAPI es el intermediario entre la base de datos PostGIS y el visor web.

API Socrata: Tecnología de publicación de datos abiertos usada por la Generalitat de Catalunya que permite consultar y filtrar *datasets* directamente desde una URL.

Backend: La parte del sistema que corre en el servidor y que el usuario no ve. En este proyecto incluye los scripts ETL en Python y la API REST con FastAPI.

BBOX (*Bounding Box*): El rectángulo que delimita lo que el usuario tiene en pantalla en ese momento. La API lo usa como filtro para devolver solo los elementos que están dentro de esa ventana geográfica.

BCIN (*Bé Cultural d'Interès Nacional*): Figura de protección del patrimonio cultural de máximo rango en Catalunya.

Clustering: Técnica que agrupa automáticamente los marcadores que están muy juntos en el mapa en un único símbolo con un número. Se abre a medida que el usuario hace zoom.

Dataset: Conjunto de datos con un esquema y una fuente comunes, tratado como una unidad. En este proyecto, las capas de patrimonio y agenda almacenadas en PostGIS.

Endpoint: Punto de acceso de una API identificado por una URL concreta. Por ejemplo, /api/patrimonio o /api/metadata.

EPSG:4326: Código del sistema de coordenadas WGS84 en grados de latitud y longitud. Es el sistema estándar para intercambio de datos geográficos en la web y el que usa todo el proyecto.

EPSG:25831: Sistema de coordenadas UTM zona 31N, el oficial en Catalunya para cartografía local. Se usa puntualmente en la auditoría de cobertura para calcular distancias métricas precisas.

ETL (*Extract, Transform, Load*): Proceso automatizado de extracción de datos desde una fuente, transformación al esquema interno y carga en la base de datos.

FastAPI: Librería de Python para construir APIs REST. La elegí porque genera documentación interactiva automáticamente y responde bien ante múltiples peticiones a la vez.

Framework: Conjunto de herramientas y código reutilizable que establece una estructura base para desarrollar aplicaciones. El visor deliberadamente no usa *frameworks* pesados de JavaScript para mantener el tiempo de carga bajo.

Frontend: La parte del sistema que el usuario ve en el navegador. En este proyecto: HTML5, CSS3, JavaScript, Leaflet.js y Turf.js.

GeoJSON: Formato de intercambio de datos geográficos basado en texto plano, estandarizado en RFC 7946. Es el formato de salida de la API REST.

IDE (*Infraestructura de Datos Espaciales*): Ecosistema de tecnologías, políticas y estándares que permite adquirir, procesar, distribuir y mejorar el uso de la información geográfica de forma interoperable.

IDEC (*Infraestructura de Dades Espacials de Catalunya*): IDE autonómica de Catalunya, coordinada por el ICGC.

IDEE (*Infraestructura de Datos Espaciales de España*): IDE nacional española, coordinada por el IGN.

INSPIRE: Directiva europea 2007/2/CE que obliga a los estados miembros a construir IDEs interoperables para la información geográfica pública.

IPAC (*Inventari del Patrimoni Arquitectònic de Catalunya*): Catálogo oficial de la Generalitat que recoge los elementos patrimoniales arquitectónicos protegidos de Catalunya.

ISO 19115: Norma internacional que define cómo debe estructurarse la documentación (metadatos) de cualquier *dataset* geográfico.

Leaflet.js: Librería de JavaScript para renderizar mapas interactivos en el navegador. Es la base del visor web de este proyecto.

NEM (*Núcleo Español de Metadatos*): Perfil español de la norma ISO 19115. Define los campos mínimos que debe cumplir cualquier *dataset* geográfico para ser integrable en la IDEE o en las IDEs autonómicas.

OGC (*Open Geospatial Consortium*): Organismo internacional que define los estándares de interoperabilidad en geoinformación, como WMS, WFS o GeoJSON.

Open Data: Filosofía según la cual los datos generados con dinero público deben estar disponibles para cualquiera, en formatos reutilizables y legibles por máquinas.

PostGIS: Extensión espacial de PostgreSQL que añade tipos de datos geométricos y funciones de análisis espacial. Permite hacer consultas como "dame todos los monumentos dentro de este rectángulo" directamente en la base de datos.

PostgreSQL: Sistema de gestión de bases de datos relacional de código abierto. Es el motor de almacenamiento del proyecto.

SPARQL: Lenguaje de consulta para bases de conocimiento estructuradas como grafos. En este proyecto se usa para interrogar a Wikidata y extraer el patrimonio monumental de Catalunya.

SOA (*Service-Oriented Architecture*): Patrón de diseño que divide el sistema en módulos independientes que se comunican a través de interfaces estándar. En este proyecto hay tres capas: datos (PostGIS), lógica (FastAPI + ETL) y presentación (visor web).

Scrollytelling: Técnica narrativa web en la que el contenido cambia o se anima a medida que el usuario baja por la página. Se usa en la introducción del visor.

ThreadPoolExecutor: Clase de Python que permite ejecutar varias tareas a la vez mediante hilos. En este proyecto gestiona las 947 consultas SPARQL al patrimonio con un máximo de 5 hilos simultáneos.

Turf.js: Librería de JavaScript que hace geoprocesamiento en el navegador, sin necesidad de llamar al servidor. En este proyecto calcula las distancias del radar de proximidad y construye las rutas culturales.

UPSERT: Operación de base de datos que combina INSERT y UPDATE. Si el registro ya existe lo actualiza; si no, lo inserta. Garantiza que ejecutar el script dos veces no genere duplicados.

Viewport: La ventana geográfica que el usuario tiene en pantalla en ese momento. Determina qué datos pide la API en cada petición.

wikibase:around: Operador del servicio SPARQL de Wikidata que devuelve todas las entidades que están dentro de un radio de distancia dado alrededor de un punto de coordenadas.

Wikidata: Base de conocimiento colaborativa y abierta de la Fundación Wikimedia. Es la fuente principal para extraer el patrimonio monumental de Catalunya en este proyecto.

WMS (*Web Map Service*): Estándar OGC que sirve mapas como imágenes renderizadas en el servidor. En este proyecto no se implementa: se optó por GeoJSON para poder filtrar dinámicamente en el cliente.

1. Introducción

1.1. Motivación y justificación: Origen y enfoque del proyecto

La elección del tema de este Trabajo de Fin de Grado nace de la inquietud que me ha acompañado durante la carrera: la sensación de que, a veces, la cartografía técnica se queda encerrada en un ámbito profesional y no conecta del todo con la sociedad. Mi intención desde el principio era desarrollar un proyecto que no terminara guardado en un cajón, sino que fuera una herramienta útil, que yo mismo quisiera utilizar y que mi entorno encontrara interesante.

El punto de inflexión fue un regalo de mi padre: el *Calendari de l'Ermità*. Este libro, que se publica desde 1875, es una referencia de la cultura popular de Catalunya, recogiendo ferias, mercados y fiestas mayores. Al leerlo, me di cuenta de que tenía ante mí una especie de puente de siglos, es decir, una base de datos analógica con un valor cultural inmenso que necesitaba ser trasladada a las herramientas actuales. Aunque la administración pública proporciona gran parte de esta información mediante iniciativas Open Data (como el portal de la Generalitat de Catalunya), estos datos se suelen encontrar dispersos, en formatos crudos (JSON, CSV) difíciles de consultar para un usuario de a pie.

A esto se suma la enorme dispersión de los datos. Actualmente es muy complicado encontrar en una misma plataforma tanto la agenda cultural de eventos como el patrimonio arquitectónico (monumentos, BCIN, etc.), lo que obliga al usuario a ir saltando de una web a otra. Además, existe un problema de visualización: la mayoría de las webs ofrecen listados de estos eventos y lugares en formato texto. Esto obliga a buscar manualmente cada ubicación, lo que rompe la experiencia del usuario. Mi visor busca eliminar esa barrera: al entrar, el sistema muestra directamente los eventos que están ocurriendo en el mapa, o permite filtrar por una fecha específica de interés de forma visual e inmediata.

Desde el punto de vista de la ingeniería, este proyecto supone un reto técnico que justifica su planteamiento como TFG. No se trata sólo de representar puntos, sino de lograr la interoperabilidad entre fuentes muy heterogéneas (consultas SPARQL a Wikidata, APIs de la Generalitat y PostGIS). A esto se suma también la dificultad de gestionar el tiempo (datos que cambian constantemente) en una plataforma profesional.

La finalidad es fomentar un turismo regenerativo: una propuesta que ayude a redistribuir el flujo de visitantes por el territorio, impulsando las economías locales y dando visibilidad digital a un patrimonio que, hasta ahora, era difícil de disfrutar de manera integrada. Para garantizar la calidad, no me he limitado a volcar datos: he conectado cada monumento con su ficha oficial de la Generalitat, uniendo la rapidez de un mapa moderno con la fiabilidad de los archivos históricos.

1.2. Objetivos

Objetivo General

El objetivo principal del trabajo no es solo hacer un mapa, sino diseñar y desarrollar una Infraestructura de Datos Espaciales (IDE) interactiva que integre en una sola plataforma el patrimonio físico (estático) con la agenda cultural y festiva (dinámica), con el fin de

promover la cultura local y fomentar la descentralización del turismo mediante tecnologías de datos abiertos y SIG de código abierto. Para lograrlo, me he marcado estos retos técnicos:

Objetivos Específicos

1. **Automatizar la captura de datos (ETL):** Programar scripts en Python para conectarme a las APIs de la Generalitat (Socrata). El reto aquí es extraer, limpiar y normalizar datos de diferentes fuentes: campos truncados, categorías compuestas y coordenadas que llegan en formatos distintos, para que todo conviva bajo el mismo esquema en PostGIS.
2. **Montar una base de datos espacial profesional:** Configurar PostgreSQL con PostGIS para que sea el motor del proyecto. No se trata solo de guardar los puntos, sino asegurar que toda la geometría use el sistema oficial (EPSG:4326) y que las consultas sean rápidas.
3. **Gestionar el "cuándo" además del "dónde":** Desarrollar el *frontend* [2]con *Leaflet*. El objetivo es que cualquier usuario pueda filtrar por fecha o proximidad de forma visual, usando técnicas como el *clustering*[3] para que el mapa no se colapse

1.3. Alcance y metodología

Este proyecto abarca toda Catalunya y se centra en el ciclo de vida completo del dato: desde que lo buscamos en el portal de *Open Data*[4] hasta que el usuario lo ve en su pantalla. No voy a generar datos nuevos yendo a campo, sino dar valor a los datos oficiales que ya existen, asegurando que sean interoperables y fáciles de consultar. Para organizar el trabajo, he seguido un proceso por fases en el que cada etapa ha alimentado a la siguiente y donde los errores y aproximaciones fallidas han formado parte del aprendizaje.

- **Fase 1 - Diseño y "fontanería" de datos:** Se ha definido la arquitectura de la base de datos y he desarrollado los scripts de extracción, transformación y carga (ETL) para las dos fuentes principales del proyecto. Esta fase también incluye algunos intentos previos que no prosperaron, como el intento de extracción mediante *web scraping*[5] de la agenda cultural, que queda documentado como parte del trabajo.
- **Fase 2 - Automatización y base de datos espacial:** He configurado el esquema PostGIS con sus tablas, índices espaciales y sistema de referencia. Además, se ha implementado una *API REST* [6]con *FastAPI* para servir los datos al visor en formato *GeoJSON*, filtrado por *bounding box* (BBOX)[7].
- **Fase 3 – Visor WebGIS y experiencia de usuario:** He construido el *frontend* completo, que integra una narrativa *scrollytelling*[8] de introducción con un panel de

[2] El frontend es la capa de presentación del sistema: lo que el usuario ve e interactúa directamente en el navegador.

[3] Agrupación visual automática de marcadores geográficos cercanos entre sí en un único símbolo cuando el nivel de zoom es bajo, con el fin de evitar la superposición y mejorar la legibilidad del mapa.

[4] Paradigma según el cual los datos generados por instituciones públicas deben estar disponibles para cualquier persona en formatos reutilizables, legibles por máquinas y sin restricciones de uso.

[5] Técnica de extracción automática de datos de páginas web mediante código, simulando la navegación de un usuario.

[6] Interfaz de programación que permite la comunicación entre sistemas a través del protocolo HTTP mediante peticiones estandarizadas, devolviendo los datos en formatos como GeoJSON.

[7] Región rectangular definida por dos coordenadas geográficas (esquina inferior izquierda y superior derecha) que delimita el área visible en pantalla en cada momento.

[8] Técnica narrativa web en la que el contenido (texto, mapas, gráficos) cambia o se anima a medida que el usuario desplaza la página hacia abajo.

control de exploración libre. Entre las funcionalidades del visor destacan los filtros dinámicos, *clustering* de marcadores, geolocalización del usuario, análisis de proximidad con *Turf.js* y un generador automático de rutas culturales.

- **Fase 4 - Documentación y validación:** He redactado esta memoria y validado el sistema: auditoría de cobertura espacial, verificación de la calidad del dato y comprobación del cumplimiento normativo respecto al NEM e INSPIRE.

2. Marco Teórico

Antes de entrar en detalle de cómo está construido el sistema, creo que vale la pena dedicar un capítulo a explicar sobre qué bases teóricas se apoya. No es un requisito de la memoria, pero muchas decisiones técnicas que he tomado a lo largo del trabajo solo tienen sentido si se entiende el contexto detrás de ellas. En este capítulo repaso los siguientes conceptos clave: qué es una IDE y por qué es el modelo adecuado para este proyecto, qué estándares gobiernan sobre la interoperabilidad y los datos geográficos y qué significa realmente construir un portal de Datos Abiertos (*Open Data*).

2.1. Las Infraestructuras de Datos Espaciales (IDE)

Una IDE no es un programa ni una herramienta concreta, sino un ecosistema. Bernabé y López la definen como el conjunto de tecnologías, políticas, estándares y recursos humanos necesarios para adquirir, procesar, almacenar, distribuir y mejorar el uso de la información geográfica (Bernabé, 2012). En la práctica una IDE es lo que permite que la información geográfica pública llegue a cualquier persona de forma estandarizada.

La analogía que mejor funciona para explicarlo es la de la red eléctrica. Cuando alguien enchufa un dispositivo no necesita saber de dónde viene la electricidad ni cómo llega. Solo necesita que el enchufe sea estándar y que la corriente funcione. Una IDE opera igual: estandariza el "enchufe" para que cualquier aplicación, institución o ciudadano pueda consumir información geográfica sin preocuparse por su origen o formato.

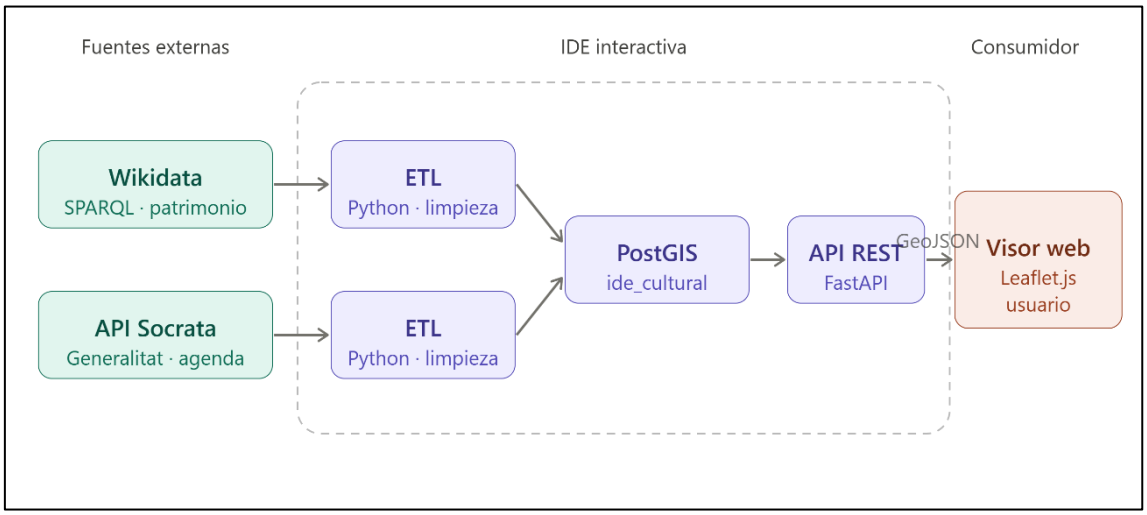


Figura 1: Esquema conceptual de la arquitectura del sistema

En este proyecto la IDE que he diseñado es ese punto de conexión. Integra datos procedentes de la Generalitat de Catalunya, de Wikidata y de la propia base de datos PostGIS y los sirve de forma unificada al visor web a través de una API REST. El usuario final no es consciente de que detrás hay tres fuentes distintas con estructuras completamente diferentes. Solo ve el mapa.

Este modelo tiene un respaldo institucional claro. A escala europea la Directiva INSPIRE (Parlamento Europeo, 2007) obligó a los estados miembros a construir sus propias IDE para garantizar la interoperabilidad de la información geográfica pública. En España esto dio lugar a la IDEE y en Catalunya a la IDEC. Mi plataforma no pretende sustituir estas infraestructuras sino ser compatible con ellas: los metadatos que genero siguen el perfil del Núcleo Español de Metadatos (NEM) (Consejo Superior Geográfico, 2010) para que los recursos que publico puedan integrarse en ese ecosistema más amplio.

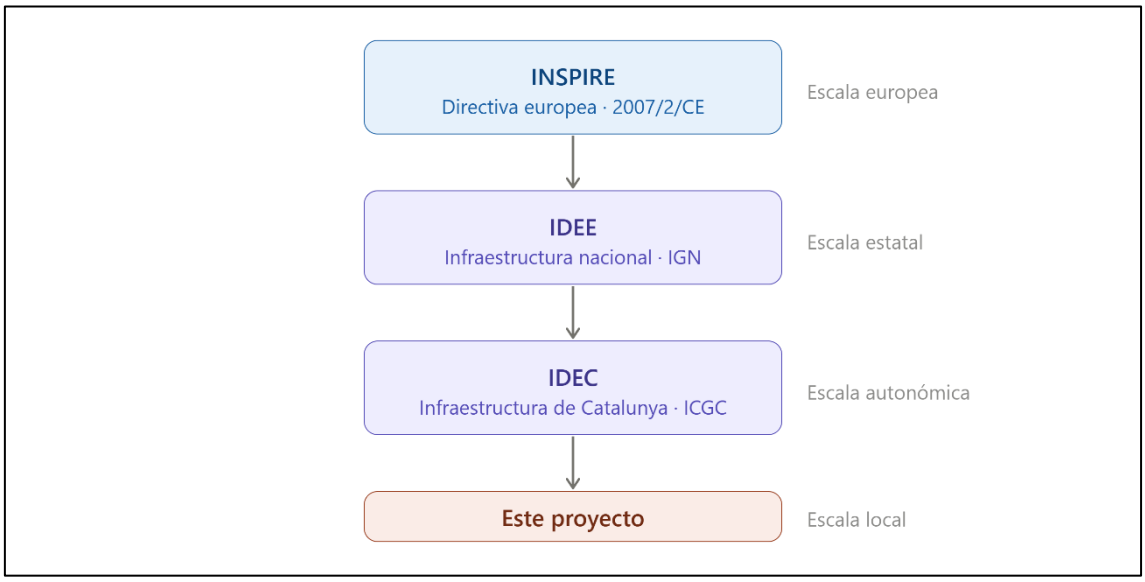


Figura 2: Jerarquía institucional de referencia

2.2. Arquitectura Orientada a Servicios (SOA)

Para que una IDE funcione en la práctica no basta con tener los datos bien almacenados. También hace falta una forma de organizarlos y servirlos que sea robusta, escalable y fácil de mantener. El patrón de diseño que mejor responde a estos requisitos es la Arquitectura Orientada a Servicios (SOA) (Erl, 2005).

La idea central de SOA es sencilla: en lugar de construir el sistema como un único bloque de código que lo hace todo, se divide en módulos independientes, cada uno con una responsabilidad concreta, que se comunican entre sí a través de la red mediante interfaces estandarizadas (W3C, 2004).

En mi proyecto este patrón se traduce en tres capas bien diferenciadas:

1. **La capa de datos,** donde la información reside y se estructura. He usado PostgreSQL con la extensión espacial de PostGIS que me permite no solo almacenar geometrías sino ejecutar directamente en el servidor consultas espaciales complejas, como filtrar los monumentos dentro del área visible en pantalla en todo momento.
2. **La capa lógica** es la maquinaria que conecta las fuentes externas con la base de datos, y la base de datos con el visor. La componen los scripts ETL en Python, que extraen y limpian los datos, y la *API REST* construida con *FastAPI* que los sirve al exterior en formato *GeoJSON*.
3. **La capa de presentación** es lo que el usuario ve y toca: el visor web construido con *Leaflet* y JavaScript. Su función no es procesar datos pesados, sino pedírselos al servidor y representarlos en pantalla de la forma más fluida posible.

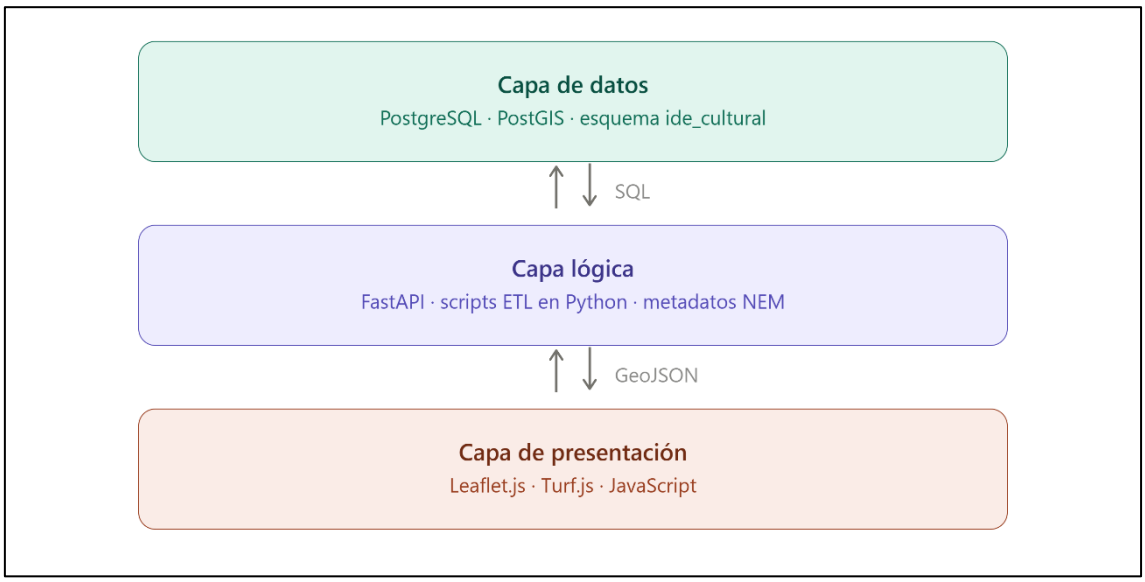


Figura 3: Arquitectura SOA del sistema en tres capas

Esta separación tiene consecuencias prácticas. Cada capa puede evolucionar de forma independiente, el sistema es más fácil de depurar cuando algo falla y la estructura sienta las bases para cumplir con los requisitos de interoperabilidad que cualquier IDE exige.

2.3. Estándares OGC y servicios de red

Si la SOA define cómo organizar el sistema internamente, los estándares del Open Geospatial Consortium (OGC) definen cómo debe hablar con el exterior (Open Geospatial Consortium (OGC), 2023). El OGC es el organismo internacional que establece las reglas en el mundo de la geoinformación: qué formatos usar, cómo estructurar sus peticiones y cómo garantizar que un mapa construido en Barcelona pueda ser consumido por una aplicación en Berlín sin que nadie tenga que traducir nada.

El estándar más conocido es el Web Map Service (WMS), que funciona enviando al cliente imágenes del mapa según la zona visualizada, renderizadas previamente en el servidor. Es el modelo que utilizan la mayoría de los visores institucionales, incluida la IDEC. Para este proyecto he optado por algo diferente: en lugar de montar un servidor WMS clásico como

GeoServer, he implementado una API REST propia con FastAPI que sirve los datos en formato GeoJSON.

La razón es puramente de rendimiento. Un WMS envía imágenes, que son archivos pesados y estáticos: cada vez que el usuario se mueve por el mapa o cambia un filtro, el servidor tiene que regenerar y enviar una imagen nueva completa. Una *API REST* que devuelve *GeoJSON* envía únicamente las coordenadas y los atributos de cada elemento, dejando que sea el navegador del usuario quien los dibuje. Esto permite que el visor aplique filtros de fecha, cambie estilos o agrupe puntos en clústeres en tiempo real sin tener que recargar el servidor en cada interacción.

El *GeoJSON* es hoy el formato estándar para intercambio de información geográfica en la web (Butler, 2016). Está basado en texto plano, los navegadores lo interpretan de forma nativa y su estructura es lo suficientemente simple como para que cualquier librería de mapas moderna, *Leaflet* entre ellas, lo consuma directamente.

Esta decisión implica renunciar a la compatibilidad con los servicios OGC clásicos, pero dado que la prioridad del proyecto es la experiencia de usuario y la capacidad de filtrar de forma dinámica, la *API REST* es la opción más coherente.

2.4. Metadatos: el NEM y la ISO 19115

Hasta ahora he explicado cómo se organiza el sistema y cómo se comunica con el exterior. Pero hay una pregunta que cualquier IDE seria ha de poder responder: ¿de dónde vienen estos datos, quién los ha generado, cuándo se actualizaron por última vez y en qué condiciones pueden usarse? La respuesta a estas preguntas está en los metadatos.

Los metadatos son básicamente la ficha técnica de los datos geográficos. De la misma forma en la que un producto lleva en su etiqueta los ingredientes, la fecha de caducidad y las instrucciones de uso, un archivo de mapas debe incluir un documento de identidad que certifique su origen, su precisión y sus condiciones de uso. Sin esa ficha los datos pueden ser técnicamente correctos pero ninguna organización externa puede evaluar si son fiables o compatibles con los suyos.

El estándar internacional que define cómo debe estructurarse esta ficha es la norma ISO 19115 (ISO, 2014). Ésta establece un modelo de metadatos con campos obligatorios y opcionales que cubren desde la extensión geográfica del *dataset*[9] hasta el sistema de referencia de coordenadas empleado, pasando por la organización responsable y la frecuencia de actualización. En España este estándar se adapta a través del Núcleo Español de Metadatos (NEM) (Consejo Superior Geográfico, 2010), que define el mínimo de campos ISO 19115 obligatorios para cualquier *dataset* que quiera integrarse en la IDEE o en las IDE autonómicas como la IDEC.

En este proyecto apliqué el NEM de dos formas. Primero documenté las dos capas principales del sistema, el patrimonio monumental y la agenda cultural, bajo la norma ISO 19115, detallando su extensión geográfica, el sistema de referencia (EPSG:4326), la fuente original y cada cuánto se actualizan. Segundo creé un endpoint específico en la *API REST*

[9] Conjunto de datos estructurados con un esquema y una fuente comunes, tratado como una unidad de información.

(`/api/metadata`) que expone estos metadatos directamente, de forma que cualquier catálogo externo puede descubrirlos y leerlos.

El resultado es que la plataforma deja de ser un proyecto académico cerrado y pasa a ser potencialmente integrable con la infraestructura de datos oficial de Catalunya.

2.5. Open Data y Turismo Regenerativo

Toda la arquitectura técnica que he descrito en los apartados de antes no tendría ningún sentido si no tuviéramos datos. Estos datos no los he generado yo yendo a campo, sino que ya existían, el problema es que existen de una manera dispersa, en formatos que eran difíciles de acceder y que no disponían de ninguna capa de visualización que los hiciera útiles para el ciudadano de a pie. Aquí es donde entra el paradigma del Open Data.

El Open Data, o datos abiertos, es la filosofía según la cual ciertos datos, especialmente los generados por instituciones públicas con dinero público, deben estar disponibles para cualquier persona en formatos reutilizables y sin restricciones de uso (Open Knowledge Foundation, 2015). El verdadero valor de este modelo no es simplemente que la información sea gratuita, sino que se ofrezca en formatos estructurados y legibles por máquinas, como las APIs Socrata que consume este proyecto, lo que permite que terceras personas podamos construir servicios sobre ellos sin depender de que la institución original los presente de una manera concreta.

En el caso de Catalunya, la Generalitat publica a través del portal de Dades Obertes una cantidad considerable de información cultural, entre ella la agenda de eventos y el inventario de patrimonio arquitectónico protegido. Sin embargo, estos *datasets* normalmente se presentan como tablas de datos crudos en formato JSON o CSV, sin ninguna capa de visualización. Mi plataforma lo que hace es actuar como un puente que toma estos datos, los proyecta sobre el territorio y los hace consultables de forma visual e inmediata.

Pero el propósito de todo esto va más allá de la parte técnica. El objetivo final de hacer accesible este patrimonio es contribuir a lo que últimamente se llama turismo regenerativo (Bellato, 2022). A diferencia del modelo turístico tradicional que se centra en maximizar el número de visitantes en los destinos ya consolidados, el turismo regenerativo busca redistribuir los flujos e impulsar las economías locales dando visibilidad a una cultura de proximidad que normalmente queda fuera del circuito.

Actualmente Catalunya concentra una parte desproporcionada de su turismo en el área metropolitana de Barcelona y en algunos puntos del litoral, mientras que el interior y las comarcas menos conocidas apenas aparecen en las rutas turísticas ni en las herramientas digitales (Institut d'Estadística de Catalunya (Idescat)). Un visor que permita al usuario ver qué festa major se celebra ese fin de semana a cincuenta kilómetros de su casa, o qué castillo románico hay cerca del pueblo donde va a pasar el puente es una herramienta modesta, pero aporta a la redistribución cultural.

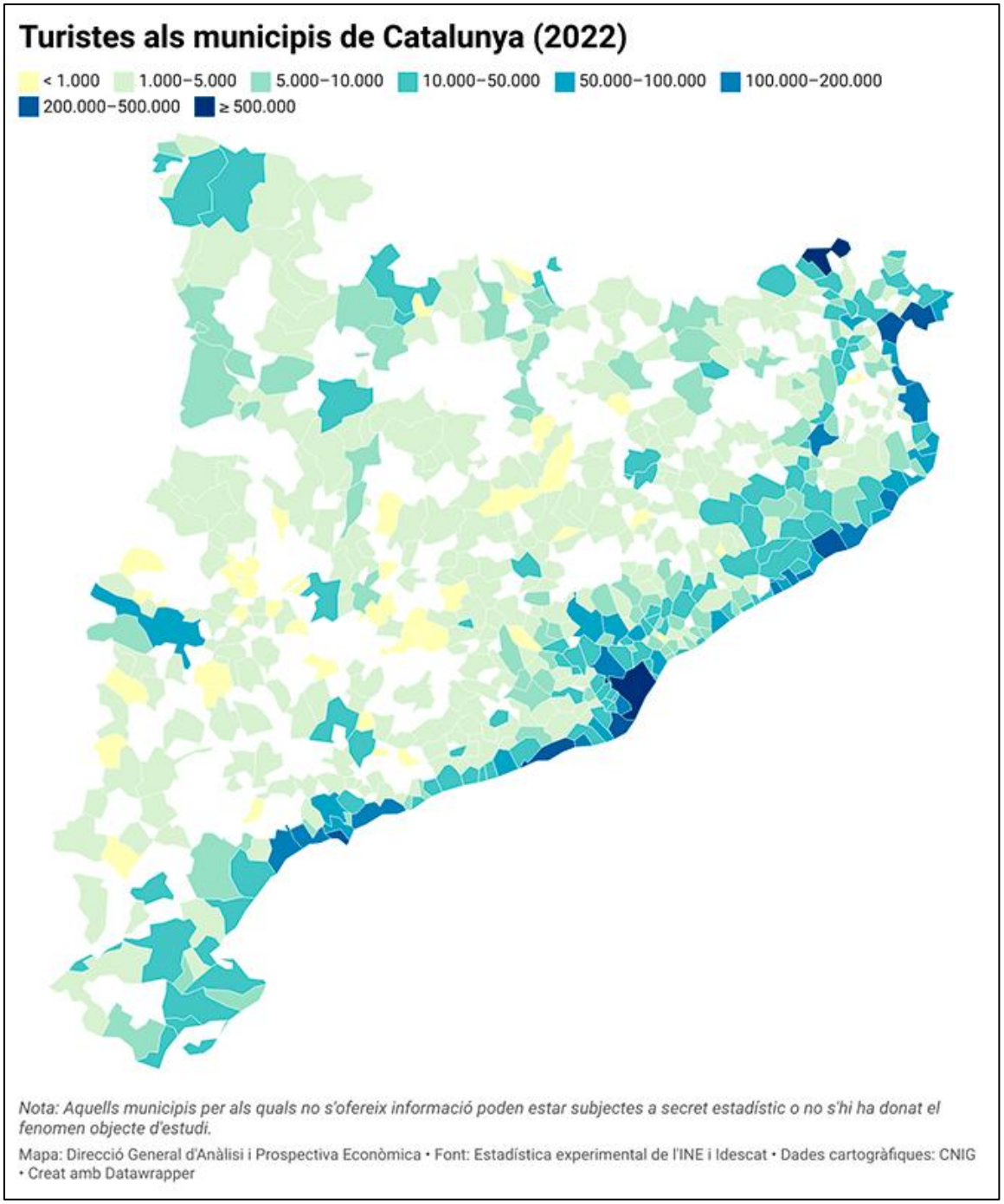


Figura 4: Distribución de turistas por municipio en Catalunya (2022) (Direcció General d'Anàlisi i Prospectiva Econòmica, 2023)

Es en este punto donde el proyecto conecta con su motivación original: el Calendari de l'Ermità como punto de partida analógico, y la ingeniería geomática como una herramienta para llevarlo al siglo XXI.

3. Diseño del sistema

3.1. Requisitos funcionales y no funcionales

Antes de ponerme a escribir una sola línea de código dediqué tiempo a definir qué debía hacer el sistema y en qué condiciones debía hacerlo. Esta distinción entre requisitos funcionales, lo que el sistema hace, y no funcionales, cómo lo hace, es la base de cualquier diseño de ingeniería serio y en una IDE es especialmente relevante.

3.1.1. Requisitos funcionales

El sistema debe cubrir seis grandes requisitos:

- **RF-01. Integración de fuentes heterogéneas.** El visor tiene que consumir simultáneamente dos fuentes de naturaleza completamente distinta: el patrimonio monumental, estático y extraído mediante SPARQL desde Wikidata, y la Agenda Cultural, dinámica y obtenida desde la API Socrata de la Generalitat de Catalunya. Ambas capas han de unirse en un mismo formato para guardarse juntas en PostGIS.
- **RF-02. Filtrado espacial por BBOX.** La API ha de aceptar como parámetros de entrada las coordenadas del *bounding box* visible en pantalla en cada momento, devolviendo únicamente los elementos dentro de esta ventana geográfica. Este requisito nace por una decisión de eficiencia: cargar todo el *dataset* en cada petición es inviable e ineficiente.
- **RF-03. Filtrado temático y temporal en el cliente.** El visor ha de permitir al usuario filtrar por texto libre, categoría y fecha sin hacer nuevas peticiones al servidor. Una vez se han cargado los datos en el *viewport*[10], toda la lógica de filtrado debe ejecutarse localmente en el navegador para garantizar una respuesta inmediata.
- **RF-04. Análisis de proximidad.** El sistema ha de ser capaz de calcular, en el lado del cliente, la distancia geodésica entre la posición GPS del usuario y cada elemento del mapa visible, devolviendo los diez elementos mas cercanos ordenados por distancia.
- **RF-05. Generación automática de rutas culturales.** A partir de la ubicación del usuario y los elementos visibles, el sistema debe construir una ruta óptima encadenando hasta seis paradas, representarla como polilínea sobre el mapa y presentar el listado de paradas.
- **RF-06. Exposición de metadatos.** La API debe publicar un *endpoint*[11] `/api/metadata` con los metadatos del *dataset* estructurados según el NEM, haciendo el sistema potencialmente integrable con catálogos institucionales como el de la IDEC.

[10] Área rectangular visible en pantalla en un momento dado. En el contexto de un visor de mapas, equivale a la ventana geográfica que el usuario está consultando.
[11] Punto de acceso concreto de una API, identificado por una URL, que responde a un tipo específico de petición y devuelve un recurso determinado.

3.1.2. Requisitos no funcionales

Los requisitos no funcionales han sido igual de determinantes en las decisiones arquitectónicas que tomé durante el proyecto:

- **RNF-01. Rendimiento y latencia.** El sistema tenía que garantizar una respuesta fluida ante las consultas de *viewport* sin que el usuario percibiera bloqueos en la interfaz. Para ello, el servidor nunca devuelve el *dataset* completo: en cada petición calcula qué elementos están dentro del área visible en pantalla y envía únicamente esos, reduciendo el volumen de datos transferidos.
- **RNF-02. Escalabilidad de la extracción.** El proceso de captura del patrimonio debe ser capaz de consultar los 947 municipios de Catalunya sin ser bloqueado por Wikidata. Para ello diseñé un sistema de peticiones concurrentes pero controladas, es decir, en lugar de lanzar todas las consultas a la vez, se mantienen activas un máximo de cinco simultáneamente y se introduce una pausa entre cada una, imitando el comportamiento de un usuario humano para no saturar el servicio externo.
- **RNF-03. Control de duplicados en la carga.** Cualquier ejecución repetida de los scripts de extracción debe producir el mismo resultado en la base de datos, sin duplicados ni datos corruptos. Dicho de otro modo: Si el mismo monumento aparece en dos ejecuciones distintas, el sistema debe actualizarlo, no duplicarlo. Esto se garantiza haciendo que cada inserción compruebe primero si el registro ya existe y, si es así, la sobrescriba con los datos más recientes.
- **RNF-04. Ligereza en el *frontend*.** El visor debe funcionar sin depender de *frameworks*[12] de JavaScript pesados que incrementaran el tiempo de carga. La apuesta por JavaScript *vanilla* junto con *Leaflet.js* y *Turf.js* fue deliberada: son librerías muy específicas que hacen una sola cosa bien, sin arrastrar funcionalidades que este proyecto no necesita.
- **RNF-05. Conformidad con estándares abiertos.** Todas las respuestas de la API han de seguir formatos reconocidos internacionalmente, de manera que los datos que se sirven sean consumibles por cualquier sistema GIS estándar, desde QGIS hasta un catálogo institucional, sin que nadie tenga que hacer ninguna traducción previa.

3.2. Modelo de datos espacial en PostGIS

El diseño del modelo de datos fue un ejercicio muy exigente porque debía acomodar en una estructura coherente información que procedía de dos fuentes con naturalezas completamente distintas: una estática y semántica, procedente de Wikidata, y otra dinámica y tabular, procedente de la API de la Generalitat. A continuación, describo las dos tablas principales.

[12] Conjunto de herramientas, convenciones y código reutilizable que establece una estructura base para desarrollar aplicaciones, reduciendo el trabajo repetitivo.

3.2.1. La tabla patrimoni_monumental

Almacena los monumentos históricos extraídos desde Wikidata. Su identificador único es el código que Wikidata asigna a cada elemento de su base de conocimiento. Usarlo como clave principal tiene una ventaja: si ejecuto el script de extracción dos veces, el sistema reconoce que un monumento ya existe y actualiza sus datos en lugar de duplicarlo.

Los campos descriptivos recogen el nombre, la descripción, la categoría del elemento, el estilo arquitectónico y el nivel de protección patrimonial, que puede ser "Bé Cultural d'Interès Nacional" (BCIN), "Bé d'Interès Local" u otras categorías del inventario. La tabla incluye además dos identificadores propios de los inventarios de la Generalitat de Catalunya: el código IPAC del inventario de patrimonio arquitectónico y el código del inventario arqueológico. Estos nos permiten construir automáticamente el enlace a la ficha oficial de cada elemento en los portales de la Generalitat, combinando la cobertura de Wikidata con la autoridad de la fuente institucional.

La tabla almacena también tres URLs de enriquecimiento pensadas para la experiencia del usuario en el visor: la imagen del elemento, el enlace a su artículo en la Wikipedia en catalán y el enlace a Google Maps para obtener indicaciones de cómo llegar. Estos campos no forman parte del núcleo de datos geográficos, pero son los que hacen que un punto en el mapa se convierta en una ficha útil para el usuario.

3.2.2. La tabla agenda_cultural

Almacena los eventos culturales dinámicos obtenidos de la API de la Generalitat. La diferencia más importante respecto a la tabla de patrimonio es que aquí los datos cambian constantemente: los eventos tienen fecha de inicio y de fin, se crean nuevos cada día y los pasados pierden relevancia. Por eso el campo de fechas es especialmente crítico: se almacena como marca de tiempo completa, lo que nos permite filtrar directamente en la base de datos qué eventos ocurren en una fecha concreta sin tener que cargarlos todos y descartarlos después.

Una decisión que merece explicación es mantener el campo `gratuita` como texto en lugar de como valor lógico verdadero/falso. La razón es práctica: la API de origen envía ese campo con valores heterogéneos: "Sí", "No", "Consultar", o simplemente vacío. Forzar una conversión hubiera implicado perder información o introducir errores. Es más honesto guardarlo tal como llega y gestionar la interpretación en el visor.

3.2.3. El esquema como unidad de organización

Ambas tablas residen bajo el mismo esquema llamado `ide_cultural` que funciona como una carpeta dentro de la base de datos. Esto mantiene todas las entidades del proyecto agrupadas y separadas del resto de tablas del servidor, lo que facilita la gestión, las copias de seguridad selectivas y una posible integración futura con infraestructuras institucionales.

3.3. Diseño de la API REST

Sin la *API REST* la base de datos sería un repositorio inaccesible y el visor no tendría datos que mostrar. La implementé con *FastAPI* por razones concretas: genera automáticamente una página de documentación interactiva en `/docs` que permite probar cada endpoint sin escribir una sola línea de código adicional, y su rendimiento ante múltiples peticiones concurrentes supera a alternativas más básicas como *Flask*. (Ramírez, 2023)

La API expone tres *endpoints*, cada uno con una responsabilidad distinta:

- `/api/patrimonio` devuelve los monumentos dentro del área visible en el mapa en ese momento. El visor le envía las coordenadas de los cuatro bordes de la pantalla y la API consulta la base de datos, que calcula internamente qué puntos caen dentro de ese rectángulo y devuelve solo esos. La respuesta también incluye un bloque de metadatos con la fuente de los datos, la fecha de generación y la licencia de uso.
- `/api/agenda` sigue la misma lógica de filtrado por área visible. El único aspecto que vale la pena mencionar es que las fechas de la base de datos llegan al visor como texto en formato estándar internacional, lo que nos permite comparar fechas directamente sin tener que hacer ninguna conversión adicional en el navegador.
- `/api/metadata` muestra los metadatos del *dataset*: quién lo ha generado, qué territorio cubre, con qué frecuencia se actualiza y en qué condiciones puede usarse. Esto hace el sistema potencialmente catalogable por infraestructuras de datos institucionales como la IDEC.

3.4. Diseño del frontend

La decisión más importante del *frontend* no ha sido elegir qué incluir sino qué descartar. El visor no usa *frameworks* adicionales: solo las librerías necesarias para la representación cartográfica y el geoprocesamiento en el cliente. Cualquier dependencia adicional incrementa el peso de lo que el navegador tiene que descargar y añade complejidad innecesaria.

3.4.1. Estructura narrativa y modo explorador

Antes de empezar a construir el visor estuve mirando cómo otros proyectos presentan información geográfica en la web. Fui a dar con dos reportajes interactivos de National Geographic que me impactaron bastante: uno sobre la construcción de las pirámides de Giza (Geographic, National, 2024) y otro sobre la expedición Franklin al Paso de Noroeste (Geographic, National, 2024). En ambos casos, a medida que bajaba por la página, el mapa se movía y cambiaba para acompañar lo que se estaba explicando. Me pareció una forma bastante efectiva e interesante de presentar el proyecto, y decidí incorporarlo.

Esa fue la inspiración directa para estructurar el visor en dos partes. La primera es una introducción narrativa: el usuario baja por la página y el mapa va acompañando lo que lee, moviéndose por el territorio a medida que avanza el relato. La segunda es el explorador libre, que arranca cuando termina la narrativa: el mapa ocupa toda la pantalla y el usuario puede navegar, buscar y filtrar.

La idea detrás de esta separación es sencilla. La narrativa pone en contexto el problema que el proyecto intenta resolver. El explorador es la herramienta. Juntos hacen que el visor funcione tanto como memoria del trabajo como algo que cualquier persona pueda usar.

Además, durante las pruebas que realicé, si no hacía esta separación, al hacer zoom en el mapa había veces que la página web se pensaba que tenía que subir a la parte narrativa. Haciendo esta separación este problema desaparece.

3.4.2. Gestión de marcadores y clustering

Uno de los problemas prácticos que aparece enseguida cuando trabajas con miles de puntos en un mapa es que, si los muestras todos a la vez, el mapa se convierte en una mancha ilegible. Para resolverlo utilizo un sistema de agrupación automática: cuando hay muchos puntos juntos en pantalla, se fusionan en un círculo con un número que indica cuántos elementos agrupa. A medida que el usuario se acerca, los grupos se van abriendo hasta mostrar cada punto individualmente.

Cada elemento se representa con un *emoji*[13] que corresponde a su categoría: una iglesia muestra ⛪, un castillo 🏰, un concierto 🎵. Me pareció más útil que un marcador genérico porque el usuario entiende de un vistazo qué tiene delante, sin necesidad de consultar ninguna leyenda.

3.4.3. Motor de filtrado en el cliente

Cuando el usuario escribe algo en el buscador, elige una categoría o cambia la fecha, el visor no va a preguntarle nada al servidor. Trabaja directamente con los datos que ya tiene en memoria y muestra solo los que encajan con lo que se está buscando. El resultado es inmediato.

Decidí que funcionara así porque cuando el usuario llega a una zona del mapa, se hace una sola petición para traer todos los elementos de esa área. A partir de ahí, mientras se queda en la misma zona, cualquier cambio de filtro es instantáneo porque los datos ya están en el navegador. No hay que esperar a ninguna respuesta del servidor.

3.4.4. Análisis de proximidad y generador de rutas

El botón de proximidad usa el GPS del dispositivo para calcular qué elementos del mapa tiene el usuario más cerca, y los presenta en una lista ordenada de menor a mayor distancia. Todo el cálculo ocurre en el navegador, sin ninguna petición adicional al servidor. (Herlocker, 2019)

El generador de rutas va un paso más allá: partiendo de donde está el usuario, va encadenando paradas eligiendo en cada momento la más cercana de las que aún no ha visitado, hasta completar un máximo de seis. La ruta se dibuja sobre el mapa como una línea discontinua y aparece también como listado en el panel lateral, con la distancia acumulada entre cada parada y la siguiente.

[13] Símbolo gráfico estandarizado incluido en el estándar Unicode, utilizado en interfaces digitales como representación visual de objetos, lugares o conceptos.

4. Implementación

En este capitulo documento cómo he construido el sistema: primero la capa de datos, luego la lógica que la alimenta y finalmente el visor que muestra al usuario la información. El orden de los apartados que voy a escribir a continuación no es aleatorio, sino que refleja el orden real en que desarrollé el proyecto, incluyendo algunos callejones sin salida que formaron parte del proceso.

4.1. Pipeline ETL de la Agenda Cultural Dinámica

El primer reto al que me enfrenté fue el de la capa de datos dinámica, es decir, los eventos culturales. A diferencia del patrimonio monumental, que cambia poco con el tiempo, la agenda cultural cambia diariamente: cada día aparecen eventos nuevos, otros caducan y las fechas se van modificando. Necesitaba un flujo de trabajo capaz de mantener la base de datos sincronizada con la realidad del territorio sin intervenir manualmente.

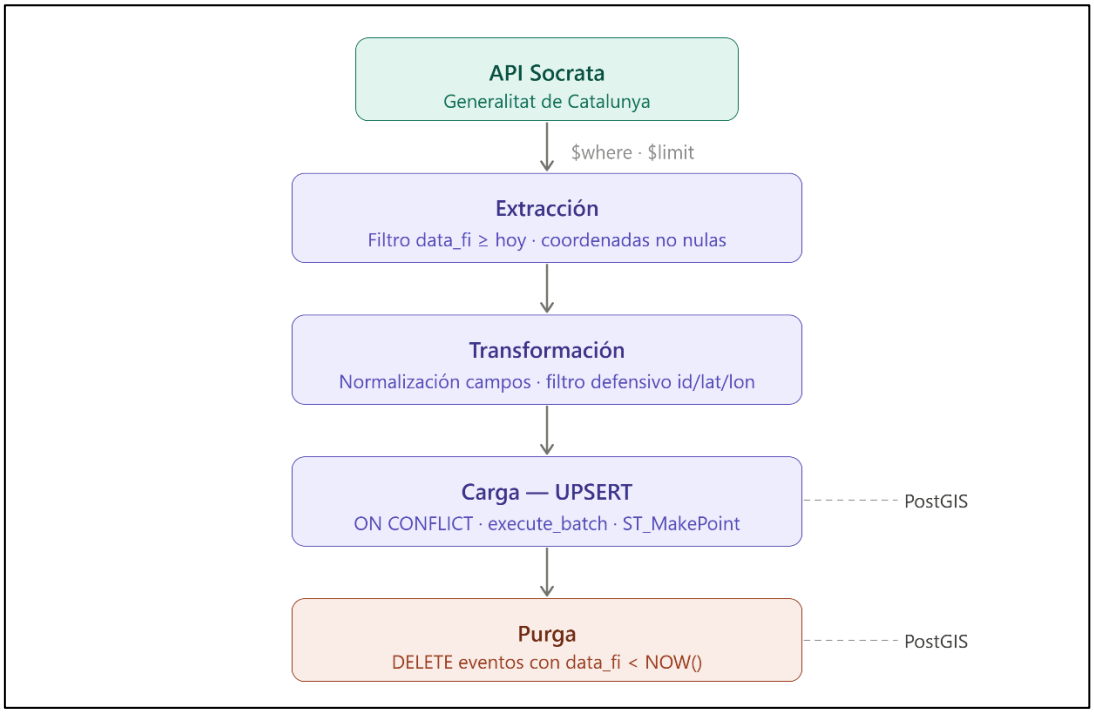


Figura 5: Pipeline ETL de la Agenda Cultural Dinámica.

El script `1_ETL_Agenda_Dinámica.py` implementa un flujo de trabajo de tres fases: extracción, transformación y carga.

4.1.1. Extracción

La fase de extracción la he resuelto con una única petición HTTP a la API Socrata. Lo que importa aquí no es la petición en sí sino el filtro que aplico antes de descargar ningún dato:

```
params = {
        "$where": f"data_fi >= '{avui}' AND latitud IS NOT NULL AND longitud IS NOT
NULL",
        "$limit": 50000
    }
```

Este filtro cumple con tres funciones. La condición `data_fi >= avui` descarta automáticamente todos los eventos que ya han finalizado, de modo que la base de datos nunca acumula registros caducados. Las condiciones `latitud IS NOT NULL` y `longitud IS NOT NULL` garantizan que solo cogemos datos que tienen coordenadas válidas, evitando que lleguen a PostGIS filas que no pueden representarse en el mapa. Y el límite de 50000 registros nos ayuda para que la fuente de datos no nos expulse cuando estemos extrayendo sus registros.

Como le pedimos a la API que haga el filtrado ella con la cláusula `$where`, en lugar de descargarlo todo y filtrarlo en local, conseguimos tener un script eficiente, ya que reducimos bastante el volumen de datos que circula por la red y el tiempo de ejecución del script.

4.1.2. Transformación

La fase de transformación convierte el dato bruto que devuelve la API al esquema interno de mi base de datos. El *dataset* de la Generalitat llega con nombres de campo truncados o con caracteres especiales, como `denominaci` en lugar de `denominacio` o `adre_a` en lugar de `adrecca`. Es una limitación habitual en bases de datos Open Data de administraciones públicas y simplemente hay que tenerla en cuenta.

El paso más importante de esta fase es cómo construimos el objeto Python que representa cada evento normalizado:

```
event = {
    "id_uuid":       item.get("codi"),
    "nom":           item.get("denominaci", "Esdeveniment sense nom"),
    "data_inici":    item.get("data_inici"),
    "data_fi":       item.get("data_fi"),
    "categoria":     item.get("tags_categor_es", "Altres"),
    "lat":           float(item.get("latitud")),
    "lon":           float(item.get("longitud")),
    ...
}
```

Antes de añadir el evento a la lista de registros válidos aplicamos un segundo filtro de seguridad: si el campo `id_uuid`, `lat` o `lon` es nulo o vacío el registro lo descarta con un `continue`. Con este doble filtrado, uno en la petición a la API y otro en la transformación local, protegemos la integridad de la base de datos frente a datos que podrían haber llegado con algún error en la fuente.

4.1.3. Carga: el patrón UPSERT

La fase de carga es donde está una de las decisiones más relevantes de este flujo de trabajo. En lugar de truncar la tabla en cada ejecución e insertar todos los registros desde cero, he implementado un *UPSERT*[14] utilizando la cláusula `ON CONFLICT` de PostgreSQL:

```
INSERT INTO ide_cultural.agenda_cultural (...)
VALUES (...)
ON CONFLICT (id_uuid)
DO UPDATE SET
    nom = EXCLUDED.nom,
    data_inici = EXCLUDED.data_inici,
    ...
    geometry = EXCLUDED.geometry;
```

La agenda cultural es un *dataset* vivo, de ahí el *UPSERT*. Si se modifica la fecha o el lugar de un evento después de que el *pipeline* ya lo haya cargado, la siguiente vez que el script se ejecute actualizará únicamente ese registro sin tocar el resto. Eliminar y reinsertar de manera masiva los datos haría que por un breve período de tiempo los datos no estuvieran disponibles cada vez que se ejecuta el *pipeline*, algo que he preferido evitar.

La carga se realiza utilizando `execute_batch` de la librería *Psycopg2*, en lugar de inserciones individuales en bucle. Esta función agrupa las sentencias SQL en lotes antes de enviarlas al servidor, lo que reduce significativamente el número de peticiones entre Python y PostgreSQL y acelera la carga cuando tenemos un volumen de datos alto.

La geometría de cada punto se genera directamente en la sentencia SQL a través de `ST_SetSRID(ST_MakePoint(lon, lat), 4326)`, sin necesidad de procesar previamente las coordenadas en Python. Así delegamos en PostGIS la construcción del objeto geométrico y nos aseguramos de que el sistema de referencia EPSG:4326 queda registrado correctamente en la columna `geometry` desde la inserción.

4.1.4. Purga de eventos caducados

Antes de extraer los nuevos registros, el script ejecuta una fase previa de limpieza. La función `purgar_events_caducats()` elimina de PostGIS todos los eventos los cuales tengan la fecha ya vencida.

```
DELETE FROM ide_cultural.agenda_cultural
WHERE data_fi < NOW();
```

Esto nos garantiza que la base de datos no va acumulando eventos caducados entre ejecuciones.

[14] Operación de base de datos que combina INSERT y UPDATE: si el registro ya existe lo actualiza, y si no existe lo inserta. Garantiza que ejecuciones repetidas del script no generen duplicados.

4.1.5. Esquema de la tabla en PostGIS

La tabla `ide_cultural.agenda_cultural` está diseñadA bajo el esquema propio `ide_cultural`, separado del esquema público de PostgreSQL. Esto facilita la gestión de permisos, permite hacer copias de seguridad selectivas y mantiene los datos del proyecto aislados del resto de información del servidor.

```
CREATE TABLE IF NOT EXISTS ide_cultural.agenda_cultural (
    id_uuid      VARCHAR PRIMARY KEY,
    nom          VARCHAR(512),
    descripcio   TEXT,
    data_inici   TIMESTAMP,
    data_fi      TIMESTAMP,
    horari       TEXT,
    gratuita     VARCHAR(50),
    espai        VARCHAR(512),
    adrecca      VARCHAR(512),
    municipi     VARCHAR(256),
    categoria    VARCHAR(256),
    imatge_url   VARCHAR(1024),
    link_entrades VARCHAR(1024),
    lat          NUMERIC(10,6),
    lon          NUMERIC(10,6),
    geometry     geometry(Point, 4326)
);
```

El campo `id_uuid` actúa como clave primaria y corresponde al campo `codi` del *dataset* de la Generalitat, que es el identificador único que la administración asigna a cada evento. Usar el identificador original de la fuente como clave primaria garantiza que el *UPSERT* sepa siempre si un evento ya existe o es nuevo con el mismo criterio que usa la fuente oficial.

4.2. Pipeline ETL del Patrimonio Estático

El segundo flujo de trabajo es técnicamente más complejo que el de la agenda. Esto es porque los datos del patrimonio monumental no los sirve ninguna API como la de la Generalitat. Al encontrarme con este problema busqué diferentes alternativas: el portal IPAC de la Generalitat no tiene una API pública, los datos del Catàleg del Patrimoni de Barcelona son de cobertura municipal y no autonómica, etc. La solución fue Wikidata. La idea me vino de una reflexión bastante simple: no puede ser que si busco la Sagrada Família en Wikipedia me aparezcan coordenadas, descripción, municipio y categoría, y que no haya ninguna manera de extraer esos datos de forma automática. Tras investigar un poco descubrí que Wikipedia y Wikidata son proyectos hermanos: toda la información estructurada que aparece en los *infoboxes*[15] de Wikipedia viene en realidad de Wikidata, y Wikidata sí tiene una API pública y un lenguaje de consulta propio. Eso era exactamente lo que necesitaba.

Wikidata es una base de conocimiento colaborativa y abierta que se mantiene gracias a la Fundación Wikimedia, la misma organización detrás de Wikipedia. A diferencia de una base de datos convencional, que organiza la información en tablas, Wikidata funciona como un grafo: cada elemento del mundo real es una entidad con un identificador único, y las relaciones entre entidades se expresan a través de propiedades. El Castell de Cardona por ejemplo, no es una fila en una tabla, sino una entidad con propiedades como "instancia de

[15] Cuadro de información estructurada que aparece en el margen de los artículos de Wikipedia, con datos clave del elemento como coordenadas, fechas o categorías.

castillo", "coordenadas geográficas", "identificador IPAC", etc. Para consultar esta estructura no sirve SQL. El lenguaje estándar para interrogar a Wikidata es SPARQL, que permite hacer preguntas como "dame todos los monumentos con coordenadas que estén dentro de un radio de 16 km de este punto". Esa es exactamente la pregunta que hace este flujo de trabajo.

La estrategia de extracción que desarrollé parte de una observación geográfica sencilla: si lanzo una consulta SPARQL con un radio de búsqueda alrededor de las coordenadas de cada municipio de Catalunya, acabaré cubriendo todo el territorio. Este enfoque no fue el primero que probé. Antes de llegar aquí intenté hacer consultas más amplias: sobre toda Catalunya de golpe o con límites de registros muy altos, pero Wikimedia las rechazaba porque el tiempo de espera se agotaba. La solución fue fragmentar el problema: en lugar de pedir todo a la vez, lanzar muchas consultas pequeñas, una por municipio, cada una con un radio de 16 km. A este mecanismo lo llamé en el código “radar”, y junto con el `time.sleep()` de 0,5 segundos entre peticiones, fue la combinación que finalmente funcionó sin que Wikidata me echara.

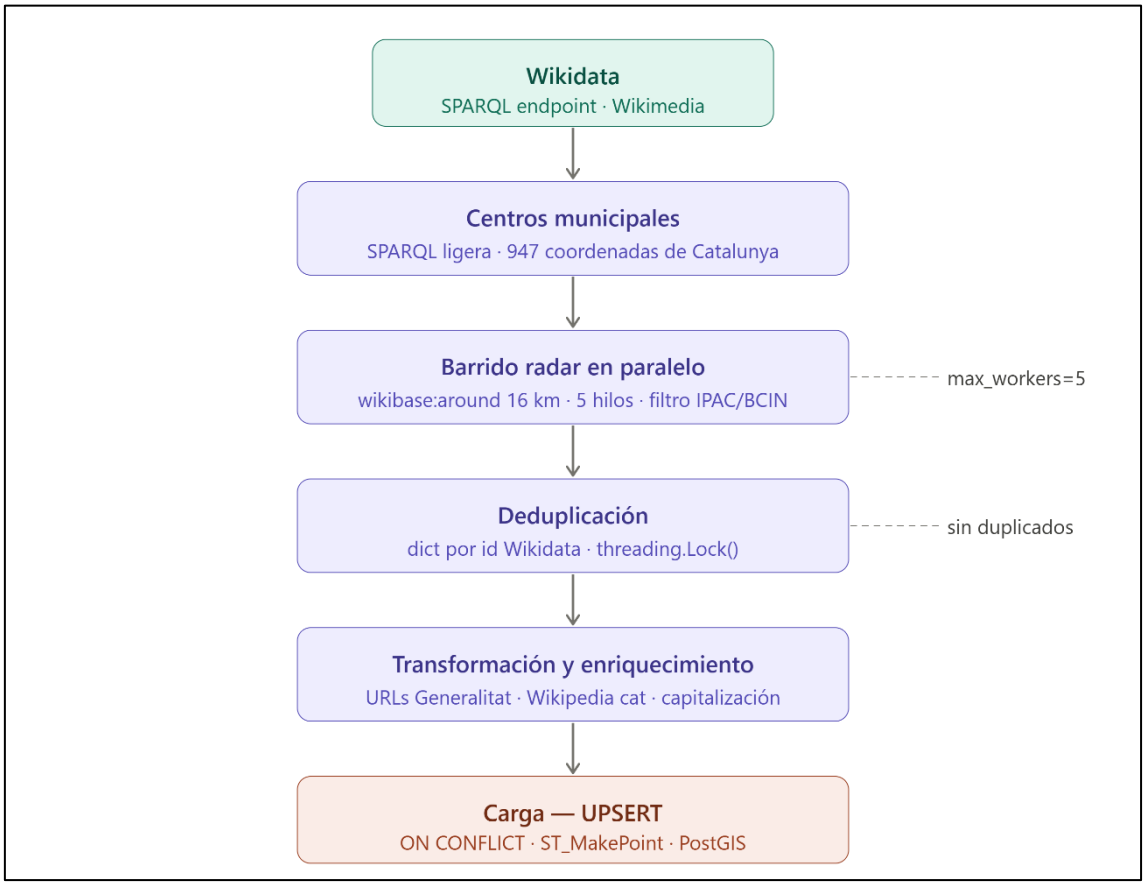


Figura 6: Pipeline ETL del Patrimonio Monumental.

4.2.1. La red de nodos: obtención de los centros municipales

El primer paso es construir la lista de puntos de todos los centros municipales de Cataluña. Para ello lanzo una consulta SPARQL ligera a Wikidata que obtiene las coordenadas de todos estos puntos sin pedir ningún atributo adicional:

```
SELECT DISTINCT ?coords WHERE {
    ?muni wdt:P31 wd:Q33146843.
    ?muni wdt:P131* wd:Q5705.
    ?muni wdt:P625 ?coords.
}
```

La propiedad `wdt:P31 wd:Q33146843` filtra entidades que son municipios de Catalunya y `wdt:P131* wd:Q5705` explora toda la jerarquía administrativa para asegurarse de que todos pertenecen administrativamente a Catalunya. El resultado es una lista de 947 coordenadas, una por municipio, que se convierten en los puntos centrales de los radares.

Pido sólo coordenadas y no nombre ni otros atributos porque el objetivo de esta consulta es únicamente construir la red de puntos lo más rápido posible. Los datos de interés vienen en la siguiente fase.

4.2.2. La consulta por radio: wikibase:around

Con los centros municipales ya extraídos, por cada punto lanzo una segunda consulta SPARQL que usa el servicio geoespacial `wikibase:around`, propio de Wikidata, para recuperar todos los monumentos en un radio de 16 kilómetros. Éste no es aleatorio: está calibrado para que no quede ninguna zona del territorio sin cubrir. Es cierto que muchos de estos radios se solapan entre ellos, pero es necesario porque algunos monumentos se encuentran en zonas rurales alejadas de cualquier núcleo urbano.

El criterio de entrada al sistema lo define un filtro UNION que cubre los principales identificadores institucionales de patrimonio reconocidos en Wikidata:

```
{
    { ?monument wdt:P1600 ?idIpac. }          # IPAC
    UNION
    { ?monument wdt:P3861 ?idArqueo. }         # Arqueològic EGIPCI
    UNION
    { ?monument wdt:P3744 ?idBpca. }           # BPCA (cobertura fronterera)
    UNION
    { ?monument wdt:P6203 ?idIbpci. }          # IBPCI
    UNION
    { ?monument wdt:P1435 wd:Q23790. }         # BCIN
    UNION
    { ?monument wdt:P1435 wd:Q15700834. }      # BCIL
    UNION
    { ?monument wdt:P1435 wd:Q23776. }         # BIC estatal
}
```

Este filtro es el que certifica la calidad del dato que conseguimos en el flujo de trabajo. Sólo entran los monumentos que estén vinculados a alguno de estos inventarios o figuras de protección oficiales: el *Inventari del Patrimoni Arquitectònic de Catalunya* (IPAC), el *Inventari del Patrimoni Arqueològic* (EGIPCI), etc. Sin este filtro Wikidata nos devolvería cualquier cosa que alguien haya etiquetado como monumento, sin ninguna validación institucional.

4.2.3. Extracción en paralelo

El problema de este enfoque es que hay que lanzar una consulta SPARQL por cada uno de los 947 municipios. Si se ejecutaran una detrás de la otra el script puede tardar varios minutos, incluso horas. Para resolverlo utilicé la librería estándar de Python `ThreadPoolExecutor` que permite lanzar varias consultas a la vez:

```
with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
    for i, centre in enumerate(centres):
        futurs.append(executor.submit(processar_node, centre, i,
                                      total_centres, monuments_dict, dict_lock))
    concurrent.futures.wait(futurs)
```

El límite de 5 consultas simultáneas, como en las decisiones anteriores, no es aleatorio. Wikidata aplica restricciones de uso que bloquean temporalmente las direcciones IP que generan demasiadas peticiones en poco tiempo. Con 5 hilos el script mantiene un ritmo de extracción rápido sin riesgo a ser bloqueado. Además, cada hilo incluye una pausa de 0,5 segundos tras cada consulta como medida adicional.

Cuando trabajamos con múltiples hilos tenemos que evitar que se pisen entre sí. Si dos hilos intentan guardar datos en el mismo diccionario exactamente en el mismo milisegundo, existe el riesgo de que la información se corrompa.

Para evitar esto, le ponemos un candado al diccionario (`threading.Lock()`). Con esto obligamos a los hilos a hacer fila: mientras uno está escribiendo, los demás esperan su turno.

```
dict_lock = threading.Lock()

with dict_lock:
    for item in resultats:
        m_id = item.get("monument", {}).get("value", "").split("/")[-1]
        monuments_dict[m_id] = item
```

Como el diccionario usa el identificador Wikidata de cada monumento como clave, si el radar de dos municipios vecinos devuelve el mismo monumento, la segunda vez que se escriba simplemente se sobrescribirá el primero con los mismos datos. De esta forma, controlamos los monumentos duplicados automáticamente.

4.2.4. Transformación y enriquecimiento

La transformación del patrimonio es más rica que la de la agenda porque aprovechamos los identificadores oficiales para construir enlaces directos a las fichas de la Generalitat:

```
if ipac:
    url_gencat = f"https://invarquit.cultura.gencat.cat/card/{ipac}"
elif arqueo:
    url_gencat = f"https://egipci.cultura.gencat.cat/ficha.jsf?id={arqueo}"
```

Esto quiere decir que cada monumento en el visor tiene un enlace directo a su ficha en el inventario oficial, y el usuario puede verificar el origen de cada dato con un solo clic. Además, si el monumento tiene un artículo en catalán, se recupera el enlace filtrando expresamente al dominio `ca.wikipedia.org` para priorizar el contenido en nuestra lengua.

Antes de guardar cada registro en la base de datos, la descripción se capitaliza automáticamente si llega en minúsculas desde Wikidata (algo muy común en este tipo de bases de datos), para garantizar una presentación uniforme en el visor.

4.2.5. Carga a PostGIS

La fase de carga replica el mismo patrón *UPSERT* del flujo de trabajo de la agenda, con `ON CONFLICT (id_wikidata)` como mecanismo de actualización. La geometría se construye de la misma manera en la misma sentencia SQL mediante `ST_SetSRID(ST_MakePoint(lon, lat), 4326)`, delegando al motor de PostGIS la generación del objeto geométrico en EPSG:4326. La tabla patrimoni_monumental convive bajo el mismo esquema que la agenda cultural (`ide_cultural`).

4.3. Auditoría de cobertura espacial

Una vez completados los dos scripts para conseguir los datos, me surgió una duda: ¿mi estrategia radar cubre realmente todo el territorio catalán, o hay zonas donde el proceso ETL no ha llegado? Para responder a esa pregunta de forma objetiva, desarrollé un tercer script, `3_QA_Cobertura_Radar.py`, cuya única función es generar un *GeoJSON* que permita visualizar el alcance real del radar.

La lógica es simple: si cada nodo del radar barre un radio de 16 km, la cobertura total del sistema equivale a la unión de círculos de 16 km centrados en cada municipio de Catalunya. Para construir esos círculos, aproveché que ya tenía en PostGIS la tabla `ide_cultural.cat_municipis` con las geometrías reales de los 947 municipios, y delegué el cálculo del buffer directamente a la base de datos mediante ST_Buffer.

La consulta es la siguiente:

```
SELECT
    nom_oficial,
    comarca,
    ST_AsGeoJSON(
        ST_Buffer(
            ST_Transform(geom_poligon, 4326)::geography,
            16000
        )::geometry
    ) AS buffer_geojson
FROM ide_cultural.cat_municipis;
```

El único detalle técnico relevante aquí es que la geometría de los municipios está almacenada en EPSG:25831 (UTM). `ST_Transform` la lleva a EPSG:4326, y el `::geography` es lo que garantiza que los 16.000 metros se calculen sobre la superficie esférica real y no como distancia plana. PostGIS hace todo esto internamente: el script Python simplemente lanza la consulta, recoge los resultados y los vuelca en un fichero GeoJSON.

El resultado es el fichero `cobertura_radar_16km.geojson` con 947 polígonos, uno por cada municipio. En la Figura 7 podemos comprobar visualmente si quedan huecos entre buffers:

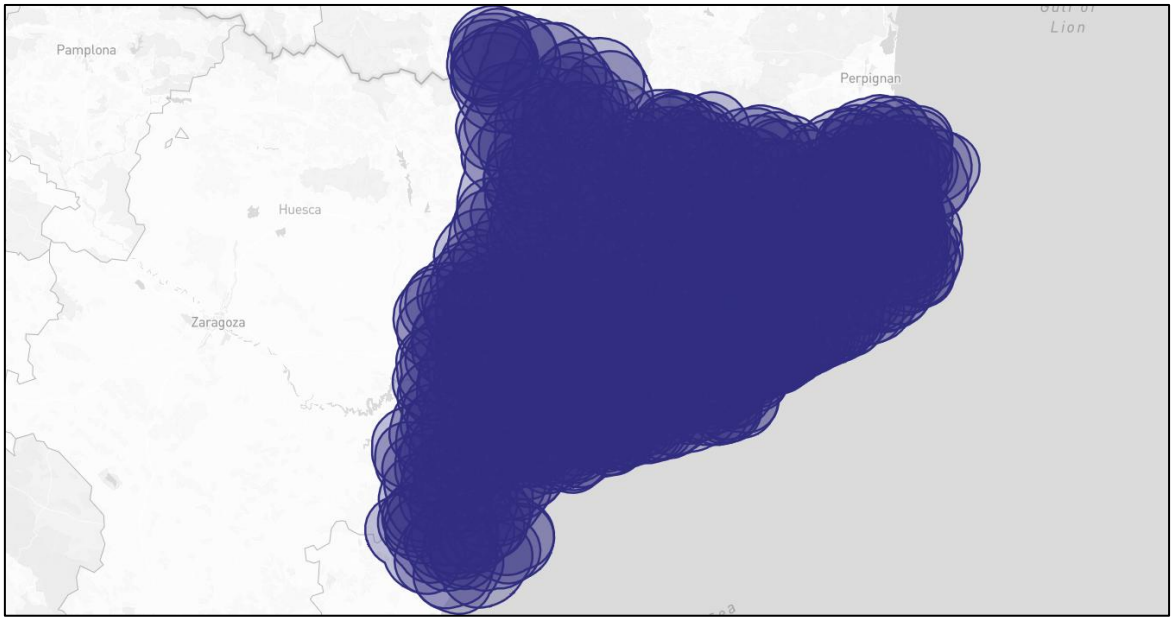


Figura 7 - Mapa de cobertura del radar ETL: 947 buffers de 16 km sobre Catalunya

Como podemos ver no existen zonas ciegas: los buffers se solapan incluso en las comarcas con municipios más alejados entre sí, como el Pallars Sobirà o la Cerdanya. El radio lo fijé para que fuese lo suficientemente grande para cubrir el interior, y lo suficientemente pequeño para no generar redundancia excesiva en el litoral y el área metropolitana.

4.4. Implementación del visor web

La capa de presentación del sistema es el visor web: la interfaz que conecta todo el trabajo anterior con el usuario final. Hay aclarar que el desarrollo del *frontend* no forma parte del núcleo de mi titulación, y que esta capa ha sido construida con asistencia de herramientas de inteligencia artificial. Lo importante aquí es justificar las decisiones de criterio que tomé: qué debía hacer el visor, qué librerías usar y cómo debía encajar todo.

4.4.1. Leaflet.js como librería de renderizado

Para renderizar el mapa elegí *Leaflet.js* frente a otras alternativas como *OpenLayers* o *MapLibre*. El motivo principal tiene que ver con el diseño del visor: quería que el mapa actuara como fondo mientras el usuario va bajando por el *scrollytelling*, volando a distintas localizaciones a medida que el usuario avanza por el texto. *Leaflet* permite este tipo de animaciones de forma muy sencilla y está muy bien documentado, lo que lo convierte la mejor opción para alguien que no viene de un perfil de desarrollo web. Además, durante las pruebas iniciales fue la que mejor rendimiento me ofreció con el volumen de datos del proyecto, aunque no hice un análisis de las causas. *OpenLayers* y *MapLibre* son más potentes, pero también más complejos, y sus ventajas no estaban justificadas para este proyecto.

4.4.2. Turf.js para el geoprocesamiento en cliente

El visor tiene dos funcionalidades que requieren cálculos geométricos: un radar de proximidad que lista los elementos más cercanos a la posición GPS del usuario, y un generador automático de rutas culturales. Para ambos usé *Turf.js*, una librería de geoprocesamiento que ejecuta los cálculos directamente en el navegador.

La alternativa habría sido montar *endpoints* de análisis espacial en el servidor *FastAPI*, pero eso habría añadido latencia de red en cada interacción del usuario y complejidad innecesaria al *backend*[16]. Si los datos ya están en memoria tras la carga por *viewport*, calcular distancias en el cliente es instantáneo y no genera ninguna petición adicional.

4.4.3. Decisiones de experiencia de usuario

Más allá de las librerías tomé dos decisiones sobre cómo debía verse el sistema. La primera fue introducir una narrativa que presentara el visor al usuario antes de llegar a él: avanza por secciones temáticas mientras el mapa vuela a localizaciones relevantes, lo que da contexto del proyecto antes de abrir el panel de control. La segunda fue mostrar los eventos del día actual cargados por defecto al entrar. De esta forma nada más entrar al visor tiene esta información a mano sin tener que configurar nada.

Además durante las pruebas detecté un problema de experiencia de usuario: estando en el visor, la rueda del ratón podía mover la página hacia arriba y reactivar accidentalmente la narrativa rompiendo la experiencia. La solución fue separar los dos modos: al entrar al visor, la narrativa desaparece completamente de la pantalla y el *scroll*[17] de la página queda bloqueado, de forma que la rueda del ratón solo controla el zoom del mapa.

4.5. Aproximaciones descartadas y decisiones de rediseño

En este último apartado documento los intentos que no llegaron a ser útiles. Lo incluyo porque el proceso de ingeniería pocas veces es lineal, y algunas de las decisiones más importantes de este proyecto surgieron precisamente de los callejones sin salida que me encontré.

4.5.1. Primeras versiones del ETL

Los scripts que han llegado al producto final no son los primeros que escribí. La primera versión del ETL de la agenda usaba *GeoPandas* y *SQLAlchemy*, herramientas perfectamente válidas pero que eran más complejas de lo necesario. Con el tiempo decidí utilizar *Psycopg2*, que conecta directamente con PostgreSQL, y decidí mantener todo en EPSG:4326 desde el principio, eliminando una transformación que no aportaba nada ya que tanto la API como *Leaflet* trabajan de forma nativa en ese sistema de referencia. El primer ETL de patrimonio procesaba los municipios uno a uno guardando la salida en un fichero *GeoJSON* temporal. Cuando introduje el multihilo con `ThreadPoolExecutor` y la carga directa a PostGIS, el tiempo de extracción bajó de forma considerable y el proceso quedó mucho más limpio.

[16] El *backend* es la capa lógica que procesa los datos en el servidor y los sirve al *frontend*.

[17] Acción de desplazar el contenido de una página web verticalmente mediante la rueda del ratón o el gesto táctil equivalente.

4.5.2. Buscando las fuentes correctas

Antes de decidir qué datos usar desarrollé un script que descargaba el catálogo completo del portal Socrata de la Generalitat y lo filtraba por temática cultural y por si disponía de coordenadas. Era una forma de no ir a ciegas: quería saber qué había disponible antes de comprometerme a utilizar ninguna fuente. De ese análisis salió la confirmación de que el *dataset* `rhpv-yr4f` de la Agenda Cultural era la opción más completa y actualizada de las que disponía.

4.5.3. El intento del Patrimoni Festiu de Catalunya

Este fue el callejón que más tiempo me costó aceptar. La idea era integrar el Patrimoni Festiu de Catalunya, un portal de la Generalitat que cataloga miles de fiestas tradicionales. Si conseguía extraer esos datos tendría una capa de eventos culturales mucho más rica que la agenda dinámica.

Monté tres scripts encadenados para intentarlo. El primero usaba *Playwright* para navegar por el portal automáticamente y extraer los títulos y URLs de cada fiesta por categoría. El segundo auditaba la calidad de los datos extraídos. El tercero intentaba estructurar la información incompleta usando GPT-4o: le pasaba el texto de cada ficha y le pedía que extrajera municipio, fechas y lugar en formato JSON para cargarlo en PostGIS. Era una tarea tan sencilla que en teoría no podía fallar.

No funcionó, y el problema no era técnico sino de origen. El portal del Patrimoni Festiu documenta las fiestas como patrimonio cultural, no como agenda de eventos: la mitad de las fichas no tenían fechas concretas ni lugar definido. La IA no podía inventarse lo que no existía, y el resultado era un *dataset* con demasiados campos vacíos para cargarlo con un mínimo de garantía de calidad.

Decidí descartarlo y quedarme con la API Socrata. Fue durante este proceso cuando descubrí algo que desconocía al empezar: la Generalitat no publica los eventos con meses de antelación, sino que los añade a medida que se acerca la fecha. Una fiesta mayor que se celebra en agosto puede no aparecer en el visor hasta semanas antes, porque es cuando la organización la publica. Eventos que ocurren cada año y que son conocidos como la Festa Major de Gràcia son invisibles en el visor durante la mayor parte del año.

Es una limitación real del sistema. La solución sería integrar el Patrimoni Festiu como capa que acompañe a la agenda, mejorando lo que intenté, de forma que el visor pudiera mostrar desde enero que en agosto habrá festa major en Gràcia. Es una de las mejoras que me gustaría abordar en el futuro.

5. Resultados y Validación

En este capítulo presento los resultados obtenidos y compruebo que funciona como se supone. No es una validación exhaustiva, pero sí una forma de documentar de manera objetiva que cada parte hace lo que tiene que hacer: que los datos son representativos, que el visor responde bien a los filtros y que los metadatos cumplen con el mínimo normativo.

Para hacerlo uso tres criterios: los datos reales de PostGIS comparados contra los repositorios de referencia oficiales, una serie de pruebas funcionales sobre el visor, y la comprobación del cumplimiento normativo respecto al NEM y al *GeoJSON*.

5.1. Resultados de la extracción de datos

Después de ejecutar los dos *pipelines* ETL, el esquema `ide_cultural` tiene dos capas con datos reales que podemos analizar.

5.1.1. Capa de patrimonio monumental

La tabla `patrimoni_monumental` tiene 31.007 registros extraídos mediante el escombrado radar sobre los 947 municipios de Catalunya. De ellos, 23.176 tienen imagen asociada (74,7%) y 7.831 no. Esto refleja la cobertura real de Wikidata: la información existe y está vinculada a un inventario oficial, pero no siempre hay fotografía disponible.

El IPAC tiene 37.503 fichas publicadas. El sistema ha capturado 31.007, lo que representa una cobertura del 82,7%. La diferencia restante corresponde a elementos del IPAC que Wikidata todavía no tiene documentados o que no están vinculados al inventario con los identificadores que el script utiliza como criterio de entrada.

	total bigint	amb_imatge bigint	sense_imatge bigint	categories_uniques bigint
1	31007	23176	7831	417

	categoria character varying (256)	total bigint		categoria character varying (256)	total bigint
1	casa	8213	16	rectoria	143
2	masia	6192	17	barraca de vinya	138
3	església	3973	18	cementiri	134
4	edifici	2740	19	capella	129
5	molí d'aigua	915	20	muralla urbana	123
6	castell	805	21	edifici escolar	115
7	edifici residencial	646	22	xemeneia de fàbrica	108
8	pont	589	23	edifici industrial	108
9	carrer	507	24	torre de sentinella	107
10	font	378	25	monument	107
11	creu de terme	354	26	pou de neu	104
12	casa consistorial	249	27	forn de calç	104
13	torre de defensa	217	28	església parroquial	102
14	plaça	216	29	passatge	97
15	casa forta	213	30	palau	96

Figura 8 - Tablas resumen de la capa patrimoni_monumental

La distribución por categoría refleja la estructura real del patrimonio arquitectónico. Las categorías más grandes son "casa" (8.213) y "masia" (6.192), seguidas de "església" (3.973) y "castell" (805).

El campo categoría tiene 417 valores distintos. Esto es normal en Wikidata: cualquier editor puede etiquetar un elemento con cualquier clase, lo que genera una cola larga de categorías muy minoritarias. El visor agrupa estas variantes con la función `normalitzarCategoria()` en el cliente, que las mapea a etiquetas generales para que cuando el usuario quiera seleccionar una categoría no le salgan 417 diferentes.

5.1.2. Capa de agenda cultural

La tabla agenda cultural tiene 958 eventos activos en el momento en el que estoy redactando este documento. Los datos se extraen del *dataset* rhpv-yr4f a través de la API Socrata de la Generalitat. Es una fuente que viene muy completa: el 100% de los eventos trae nombre, descripción, imagen y enlace. En total los 958 eventos están repartidos por 187 municipios, de los cuales 288 eventos (un 30,06%) son gratuitos.

El problema principal nos lo encontramos en las categorías. La API no devuelve etiquetas limpias, sino que junta rutas internas separadas por comas (por ejemplo: `agenda:categories/concerts, agenda:categories/dansa`). Como a un mismo evento le pueden corresponder varias categorías a la vez, nos topamos con 97 combinaciones distintas mezcladas en la base de datos.

Para no pasarle esos datos en bruto al visor hacemos la limpieza directamente en el frontend. La función `desSlug()` se encarga de coger estas cadenas, sacar el texto que realmente nos interesa y formatearlo al vuelo, de forma que el usuario vea el selector de filtros con las categorías limpias y bien escritas.

5.2. Validación funcional del visor

5.2.1. Rendimiento de la API

Para comprobar que la API responde correctamente y dentro de unos tiempos razonables, lancé unas peticiones que me devolvieron los siguientes resultados:

Petición	Tiempo de respuesta	Código HTTP	Elementos devueltos
Patrimoni bbox Catalunya	1.286 ms	200	24.815
Patrimoni bbox pequeño	201 ms	200	1.863
Agenda bbox Catalunya	103 ms	200	958
Metadata	13 ms	200	—

Como podemos ver, la estrategia de filtrado por *bounding box* funciona: cuando el bbox es grande (toda Catalunya) el servidor tarda 1,3 segundos en devolver casi 25.000 elementos, pero cuando es pequeño la respuesta baja a 200ms. En condiciones normales de uso del visor, donde el usuario nunca tiene toda Catalunya a la vista sino una zona concreta, los tiempos de respuesta están por debajo de los 300ms.

5.2.2. Filtros y carga dinámica

El comportamiento esperado era que al mover el mapa o hacer zoom, el visor hiciera una nueva petición a la API y recargara únicamente los datos visibles en pantalla. Tras las pruebas puedo confirmar que el sistema funciona correctamente: el margen de espera de 700ms evita que cada pequeño movimiento del mapa genere una petición, y la carga se ejecuta solo cuando el usuario ha dejado de mover la vista.

Los filtros de texto, categoría y fecha actúan sobre los puntos ya cargados en memoria, sin llamar a la API. Filtrar por "castell" o por una fecha concreta es instantáneo desde el punto de vista del usuario. El filtro de fecha funciona correctamente con el rango `data_inici` / `data_fi`: un evento que empiece el día 1 y acabe el día 10 aparece al filtrar por cualquier día de ese intervalo.

5.2.3. Clustering

Incluso con los 31.007 monumentos cargados a la vez en todo el mapa, la navegación es fluida. Esto confirma que fue un acierto configurar *Leaflet* con `preferCanvas: true`. Al hacerlo, el navegador dibuja los puntos directamente sobre un lienzo único en vez de crear un elemento HTML por cada marcador, lo que alivia muchísimo la carga de memoria al manejar un volumen de datos tan grande.

5.3. Conformidad normativa

5.3.1. GeoJSON y RFC 7946

Los dos *endpoints* principales (`/api/patrimonio` y `/api/agenda`) devuelven *FeatureCollections* válidas según RFC 7946 (Butler, 2016). Las geometrías salen de PostGIS mediante `ST_AsGeoJSON()` y el sistema de referencia es EPSG:4326 en todas las respuestas. Cualquier cliente GIS que consuma estos *endpoints* sabe exactamente en qué coordenadas está trabajando.

5.3.2. NEM e ISO 19115

El *endpoint* `/api/metadata` devuelve una ficha que cubre los campos obligatorios del Núcleo Español de Metadatos:

- **Identificador**: cat-ide-cultural-v1
- **Idioma**: Catalán (cat)
- **Nivel jerárquico**: *Dataset*
- **Extensión geográfica**: *Bounding box* de Catalunya en EPSG:4326 (0.159°E – 3.334°E, 40.52°N – 42.86°N)
- **Resolución espacial**: 1:50.000
- **Frecuencia de actualización**: Diaria
- **Formato de distribución**: *GeoJSON*
- **Linaje**: Wikidata vía SPARQL para el patrimonio, API Socrata de la Generalitat para la agenda, almacenadas en PostGIS en EPSG:4326
- **Estándar**: NEM — Núcleo Español de Metadatos (ISO 19115:2003)

Que los metadatos sean accesibles por un *endpoint* propio significa que cualquier catálogo externo puede leerlos automáticamente, sin que nadie tenga que buscarlos a mano.

6. Conclusiones

Este último capítulo cierra el trabajo. Repaso si los objetivos que me marqué al principio se han cumplido, reconozco los límites reales del sistema y apunto las líneas de mejora que dejaría abiertas. Termino también con una reflexión breve sobre lo que ha significado este proyecto para mí.

6.1. Evaluación de objetivos

El objetivo de este trabajo era diseñar y desarrollar una IDE interactiva que integrara patrimonio estático y agenda cultural dinámica en una sola plataforma, usando exclusivamente tecnologías de código abierto y datos públicos. Viendo el resultado puedo decir que ese objetivo se ha cumplido.

El sistema funciona: la base de datos PostGIS tiene 31.007 monumentos y una agenda conectada a la API Socrata de la Generalitat, la *API REST* con *FastAPI* sirve ambas capas en formato *GeoJSON* con tiempos de respuesta por debajo de los 300ms en condiciones normales de uso, y el visor web integra filtros dinámicos, *clustering*, geolocalización, análisis de proximidad y un generador de rutas culturales.

Sobre los objetivos que me marqué al principio:

- **Automatizar la captura de datos (ETL):** Cumplido. Los dos *pipelines* están operativos. El de la agenda extrae, limpia y carga en PostGIS los eventos activos de la Generalitat. El del patrimonio implementa una estrategia radar sobre los 947 municipios de Catalunya usando `wikibase:around` de Wikidata, con una cobertura del 82,7% respecto al IPAC.
- **Montar una base de datos espacial:** Cumplido. El esquema ide_cultural en PostgreSQL/PostGIS almacena las dos capas bajo el mismo sistema de referencia (EPSG:4326), con índices espaciales y geometrías construidas por su motor usando ST_MakePoint.
- **Gestionar el "cuándo" además del "dónde":** Cumplido. El visor filtra por fecha con un rango data_inici / data_fi, los eventos del día actual se cargan por defecto al entrar, y el filtro de texto y categoría actúa sobre los datos ya en memoria sin peticiones adicionales al servidor.

El punto sobre el despliegue en un servidor con un dominio propio queda pendiente, tal y como explico en el apartado de limitaciones.

6.2. Limitaciones del sistema

Hay tres limitaciones que considero que son importantes documentar. Más que nada porque no son errores ni descuidos: son consecuencias de decisiones que he tomado, o condicionantes externos que no podía resolver.

- **Falta de antelación de la API Socrata.** La Generalitat no publica los eventos con meses de antelación. Los añade a medida que se acerca la fecha, y la API solo devuelve los que tienen data_fi mayor o igual a hoy. Esto quiere decir que fiestas que se repiten y que son muy conocidas, como la Festa Major de Gràcia, son invisibles en el visor durante la mayor parte del año porque la organización no las publica hasta semanas antes. No es un fallo del flujo de trabajo: el sistema extrae exactamente lo que hay. Es una limitación de la fuente de datos original que no puedo arreglar.
- **El sistema solo funciona en local.** La API y el visor corren en mi máquina y no son accesibles desde fuera. Desplegar en un servidor público con dominio y HTTPS queda fuera del alcance de este trabajo, pero no es un problema técnico: la arquitectura está preparada para ese paso sin cambios estructurales.
- **La cobertura del patrimonio depende de Wikidata.** El 82,7% de cobertura respecto al IPAC es un buen resultado, pero el 17,3% restante corresponde a elementos que Wikidata no tiene documentados o que no tienen los identificadores institucionales que el script usa como criterio de entrada. El sistema no puede capturar lo que la comunidad de Wikidata todavía no ha vinculado al IPAC. Esto no es un defecto de la estrategia radar, sino un reflejo honesto del estado actual de la web semántica en lo que respecta al patrimonio catalán.

6.3. Mejoras futuras

A partir del trabajo que he hecho y de los límites que he detectado, hay tres líneas de mejora claras que dejaría abiertas:

- **Integrar el Patrimoni Festiu de Catalunya como capa complementaria.** Como he comentado anteriormente, el problema que encontré al intentar integrar esta capa era que el portal no tiene fechas concretas en muchas fichas porque documenta las fiestas como patrimonio cultural, no como agenda. Habría que plantear alguna estrategia diferente para poder solventar este problema. De esta manera el visor podría mostrar desde enero que en agosto habrá festa major en Gràcia, complementando la agenda dinámica que solo aparece semanas antes.
- **Despliegue en producción con dominio y HTTPS.** Es el paso más obvio y el que más impacto tendría. Lo que está construido ya funciona: solo falta sacarlo de mi máquina y ponerlo en un servidor.
- **Exponer servicios OGC estándar mediante *pygeoapi*.** La idea sería añadir *pygeoapi* por encima de la misma base de datos PostGIS que ya está cargada. Esto haría la plataforma compatible con clientes GIS institucionales y con el ecosistema IDEC/IDEE, sin tocar nada de lo que está construido.

6.4. Reflexión final

Como comenté en la introducción, este proyecto nació de un libro de papel que lleva publicándose desde 1875, el Calendari de l'Ermità. Al leerlo pensé: qué interesante sería tener esto en un mapa. Ver a la vez dónde está el patrimonio y qué pasa culturalmente cerca, sin tener que saltar entre webs. Existen agendas culturales y existen visores de patrimonio, pero siempre por separado. La idea era juntarlo todo.

Lo que he construido responde a esa idea. No es perfecto, tiene limitaciones claras y cosas que mejoraría, pero funciona y está hecho con criterio técnico. Para mí eso es suficiente para cerrar el trabajo con buena sensación.

Lo que más me llevo del proceso es lo que he aprendido a medida que lo hacía. Antes de empezar nunca había montado un proceso ETL, no sabía de la existencia del *web scraping* para extraer datos, ni que Wikidata tenía detrás tanto patrimonio estructurado y accesible. Descubrir que con las herramientas correctas puedo tener a mi alcance esa cantidad de datos y hacer algo útil con ellos es lo que más me ha llenado. Y documentar los intentos fallidos y los problemas que no vi venir, en lugar de presentar solo la parte que salió bien, es lo que creo que le da valor real a este trabajo como ejercicio de ingeniería.

7. Análisis de sostenibilidad e implicaciones éticas

7.1. Impacto ambiental

El proyecto se ha desarrollado en local, sobre mi equipo de sobremesa (Intel Core i7-11700F, RTX 4060), sin servidores dedicados ni infraestructura en la nube. Estimando 320 horas de trabajo a un consumo medio de 150 W, el gasto eléctrico total ha sido de aproximadamente 48 kWh, equivalentes a unos 9,1 kg de CO_2, según el factor de emisiones de la red eléctrica española de 2023 (0,19 kg CO_2e/kWh). No se ha generado ningún residuo material y el seguimiento con el tutor se ha hecho principalmente por correo, con una única reunión virtual, por lo que los costes por desplazamiento desaparecen.

Si el sistema se desplegara en producción, el impacto seguiría siendo bajo: bastaría un servidor VPS (*Virtual Private Server*), sin hardware dedicado ni instalaciones físicas.

7.2. Impacto económico

Todas las tecnologías que he utilizado son de código abierto y gratuitas. El coste real del proyecto es el de las horas de trabajo: estimando 320 horas a 20 €/hora, el desarrollo del TFG es de unos 6.400 €. No ha habido ningún gasto directo en licencias ni infraestructura.

En producción, el coste de operación sería de entre 5 y 15 €/mes por un servidor VPS básico. Al ser el código público en GitHub, cualquier entidad que quisiera reutilizarlo podría hacerlo sin coste de desarrollo adicional.

7.3. Impacto social

El impacto social potencial del visor es dar visibilidad digital al patrimonio de municipios pequeños de Catalunya que no aparecen en las rutas turísticas habituales, contribuyendo a redistribuir el turismo por el territorio. El sistema no recopila datos personales de los usuarios y no requiere registro, por lo que cumple el RGPD por diseño.

El principal riesgo social es la cobertura incompleta: el 17,3% del IPAC no está documentado en Wikidata, lo que significa que hay patrimonio real que el visor no muestra.

7.4. Implicaciones éticas

El proyecto trabaja con datos públicos bajo licencias de uso libre. No se recogen datos personales de ningún usuario. El enlace a la ficha oficial de la Generalitat en cada monumento permite al usuario contrastar la información con la fuente institucional autorizada, lo que minimiza los posibles errores por el carácter colaborativo de Wikidata.

El proyecto respeta el Código Ético de la UPC y el Código de Integridad de la Investigación, documentos firmados al formalizar la matrícula del TFG.

7.5. Relación con los ODS

El proyecto contribuye a tres objetivos de la Agenda 2030:

- ODS 11 (Ciudades y comunidades sostenibles, meta 11.4): visibilidad digital del patrimonio cultural de Catalunya.
- ODS 8 (Trabajo decente y crecimiento económico, meta 8.9): fomento del turismo sostenible y descentralización de los flujos turísticos.
- ODS 17 (Alianzas para lograr los objetivos, meta 17.18): reutilización de datos públicos y código abierto como base para servicios de valor añadido.

Bibliografía

Bellato, L. /. (2022). Regenerative tourism: a conceptual framework understanding where we are heading. *Tourism Recreation Research, 47*(5-6), 730-734.

Bernabé, M. A. (2012). *Fundamentos de las Infraestructuras de Datos Espaciales (IDE)*. UPM Press.

Butler, H. e. (2016). *The GeoJSON Format — RFC 7946*. Obtenido de https://datatracker.ietf.org/doc/html/rfc7946

Consejo Superior Geográfico. (2010). *Núcleo Español de Metadatos (NEM), versión 1.2*. Grupo de Trabajo IDEE.

Direcció General d'Anàlisi i Prospectiva Econòmica, G. d. (1 de Junio de 2023). *Turisme estranger als municipis de Catalunya a partir de telefonia mòbil*. Obtenido de Apunts d'Economia: https://apuntseconomia.blog.gencat.cat/2023/06/01/turisme-estranger-municipis-catalunya-telefonia-mobil/

Erl, T. (2005). *Service-Oriented Architecture: Concepts, Technology, and Design*. Prentice Hall.

Geographic, National. (2024). *National Geographic*. Obtenido de Rethinking how the Egyptians built the pyramids: https://www.nationalgeographic.com/history/graphics/see-how-egyptians-built-giza-pyramids

Geographic, National. (2024). *National Geographic*. Obtenido de Northwest Passage mystery: the Franklin Expedition: https://www.nationalgeographic.com/history/graphics/northwest-passage-mystery-franklin-expedition-sailing-ice

Herlocker, M. e. (2019). *Turf.js*. Obtenido de Advanced geospatial analysis for browsers and Node.js: https://turfjs.org/

Institut d'Estadística de Catalunya (Idescat). (s.f.). *Turisme. Indicadors*. Obtenido de https://www.idescat.cat/indicadors/?id=aec&n=15530

ISO. (2014). *ISO 19115-1:2014 Geographic information — Metadata — Part 1: Fundamentals*. International Organization for Standardization.

Open Geospatial Consortium (OGC). (2023). *OGC Standards and Resources*. Obtenido de https://www.ogc.org/standards/

Open Knowledge Foundation. (2015). *The Open Data Handbook*. Obtenido de http://opendatahandbook.org/

Parlamento Europeo. (2007). *Directiva 2007/2/CE por la que se establece una infraestructura de información espacial en la Comunidad Europea (INSPIRE)*. Diario Oficial de la Unión Europea.

Ramírez, S. (2023). *FastAPI*. Obtenido de High performance, easy to learn, fast to code, ready for production: https://fastapi.tiangolo.com/

W3C. (2004). *Web Services Architecture*. Obtenido de W3C Working Group Note: https://www.w3.org/TR/ws-arch/

Anexo I: Repositorio del código

El código fuente completo del proyecto está disponible públicamente en el siguiente repositorio de GitHub: https://github.com/guillemfernandezz/TFG_Guillem_Fernandez